ТОО «Колледж Хекслет»

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ

на тему: «Разработка веб-приложения «платформа для регистрации на кибертурниры» с использованием Firebase»

профессиональный модуль: ПМ3 «Разработка модулей программного обеспечения для компьютерных систем»;

Специальность: 06130100 – Программное обеспечение (по видам)
Квалификация: 4S06130105 – Техник информационных систем

Выполнил: студент 3 курса
8/23 ТИС группы
Сабыржан Алкей Гуламжанулы

Допущен к защите «____ »_____ 20 г.

Преподаватель:_____Турман Н.Р.

(подпись)

Защитил с оценкой:

_____

(буква) (циф.экв.) (балл)

Алматы 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ **5**
1.1 Выбор темы и предметная область **5**
1.2 Визуальная схема структуры Firestore **6**
1.3 Связи между коллекциями **7**

2. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ (FIRESTORE SCHEMA) **8**
2.1 Выбор темы и описание предметной области **8**
2.2 Визуальная схема структуры Firestore **9**

3. ПРОЕКТИРОВАНИЕ САЙТА **10**
3.1 Внешняя структура сайта **10**
3.2 Внутренняя структура сайта **11**

4. ПРАКТИЧЕСКАЯ ЧАСТЬ **13**
4.1 Настройка Firebase проекта **13**
4.2 Создание схемы базы данных и тестовых данных **14**
4.3 Реализация системы аутентификации **14**
4.4 Разработка главной страницы с пагинацией **16**
4.5 Реализация механизмов поиска и многоуровневой фильтрации **17**
4.6 Страница детального просмотра (TournamentDetail.vue) **18**
4.7 Система взаимодействия (Заявки на участие) **20**
4.8 Личный кабинет пользователя (Profile.vue) **21**
4.9 Реализация режима реального времени (Real-time) **22**
4.10 Админ-панель (Admin.vue) **23**
4.11 Оптимизация и правила безопасности (FIrestore Rules) **25**

ЗАКЛЮЧЕНИЕ **26**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **27**

ВВЕДЕНИЕ

Актуальность темы проекта. В современном мире киберспорт перестал быть просто развлечением и трансформировался в глобальную индустрию с многомиллионной аудиторией и сложной экосистемой. С ростом количества профессиональных и любительских турниров по таким дисциплинам, как Counter-Strike 2, Dota 2 и Valorant, возрастает потребность в автоматизированных инструментах управления этими мероприятиями. Существующие платформы зачастую перегружены функционалом или требуют высокой стоимости подписки, что создает барьер для малых организаторов и киберспортивных клубов.

Разработка веб-приложения «CYBERTOUR» обусловлена необходимостью создания гибкого, быстрого и интуитивно понятного инструмента для регистрации участников, ведения статистики и управления контентом в режиме реального времени. Внедрение подобных систем позволяет минимизировать человеческий фактор при обработке заявок, обеспечить прозрачность формирования призовых фондов и повысить вовлеченность игроков.

Выбор стека технологий. Для реализации проекта был выбран современный стек, включающий фреймворк Vue.js и облачную платформу Firebase.

1. **Vue.js** обеспечивает высокую реактивность интерфейса, что критически важно для динамического отображения статусов турниров и результатов поиска без перезагрузки страницы.
2. **Firebase (Firestore и Authentication)** позволяет отказаться от разработки сложной серверной части (Backend), предоставляя готовую инфраструктуру для безопасной авторизации пользователей и масштабируемую NoSQL базу данных, работающую в режиме реального времени. Это значительно ускоряет процесс разработки и развертывания приложения.

Целью учебной практики является систематизация и закрепление теоретических знаний, полученных при изучении модуля ПМ.03 «Разработка модулей программного обеспечения для компьютерных систем», а также применение их на практике для создания полнофункционального веб-приложения.

Для достижения поставленной цели необходимо решить следующие задачи:

Провести анализ предметной области и сформировать функциональные требования к системе организации киберспортивных турниров.

Спроектировать архитектуру базы данных Firestore, обеспечивающую оптимальное хранение информации о пользователях, турнирах и заявках.

Реализовать систему аутентификации и авторизации пользователей с разграничением прав доступа (пользователь и администратор).

Разработать пользовательский интерфейс с использованием компонентного подхода Vue 3.

Внедрить механизмы фильтрации, поиска и пагинации данных для работы с большими массивами информации.

Разработать модуль администратора для управления жизненным циклом турниров (создание, редактирование, завершение).

Настроить правила безопасности (Security Rules) для защиты данных от несанкционированного доступа.

Объектом исследования является процесс разработки Fullstack веб-приложений на базе облачных технологий, который в современных условиях приобретает особую актуальность в связи с ростом требований к масштабируемости, доступности и

производительности программных решений. В рамках данного исследования рассматриваются ключевые подходы, методы и принципы построения распределённых веб-систем, ориентированных на эффективное взаимодействие клиентской и серверной частей, а также на обеспечение высокой скорости обработки пользовательских запросов.

Предметом исследования выступают инструменты Firebase и возможности фреймворка Vue.js в контексте создания высоконагруженных интерфейсов. Особое внимание уделяется анализу их функциональных возможностей, преимуществ и ограничений при разработке современных веб-приложений. Рассматриваются такие аспекты, как организация клиентской логики, управление состоянием приложения, работа с асинхронными данными, а также интеграция с облачными сервисами. Дополнительно исследуются вопросы оптимизации производительности, повышения отзывчивости пользовательского интерфейса и обеспечения устойчивости приложения при увеличении нагрузки.

Ожидаемые результаты. Итогом работы является готовое к эксплуатации приложение «CYBERTOUR», обладающее модульной и расширяемой архитектурой, что обеспечивает удобство дальнейшей поддержки и масштабирования. Разрабатываемое решение должно соответствовать современным стандартам веб-разработки, учитывать требования к безопасности, производительности и пользовательскому опыту, а также полностью отвечать техническому заданию, изложенному в данном отчёте. Предполагается, что полученное приложение сможет эффективно функционировать в условиях реальной эксплуатации, демонстрируя стабильную работу, удобный интерфейс и высокий уровень взаимодействия с конечным пользователем.

1. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

1.1 Выбор темы и предметная область

В качестве итогового проекта в рамках учебной практики была выбрана тема **«Разработка веб-приложения CYBERTOUR с использованием облачной платформы Firebase»**. Данный выбор обусловлен стремительным развитием индустрии киберспорта и высокой востребованностью автоматизированных систем для управления турнирными процессами.

Анализ предметной области: Предметная область проекта охватывает полный цикл организации дистанционных соревнований по популярным компьютерным дисциплинам. В современных условиях организаторам требуется инструмент, обеспечивающий высокую скорость обработки данных и корректное взаимодействие между участниками в режиме реального времени.

В рамках проектируемой системы выделяются следующие ключевые бизнес-процессы:

1. **Управление пользователями:** идентификация участников, разделение прав доступа (ролевая модель) и ведение игровых профилей.
2. **Контент-менеджмент:** публикация актуальной информации о турнирах, их регламенте и призовых фондах.
3. **Регистрация и учет:** сбор и хранение заявок на участие, контроль лимитов команд и отслеживание статусов соревнований.

Основные сущности системы: Для обеспечения корректной работы приложения были выделены и структурированы следующие информационные сущности:

Пользователи (Users): база данных игроков, включающая уникальные идентификаторы, контактные данные и системные роли (Player/Admin).

Турниры (Tournaments): центральная сущность, содержащая метаданные о мероприятии (название дисциплины, дата начала, максимальное количество участников, текущий статус).

Регистрационные формы (Registrations): транзакционные сущности, фиксирующие факт участия конкретного пользователя в определенном турнире и связывающие воедино данные игрока и соревнования.

Использование облачной архитектуры Firebase (BaaS) в данной предметной области позволяет минимизировать задержки при обновлении списков участников и гарантирует целостность данных при одновременной подаче большого количества заявок.

1.2 Визуальная схема структуры Firestore

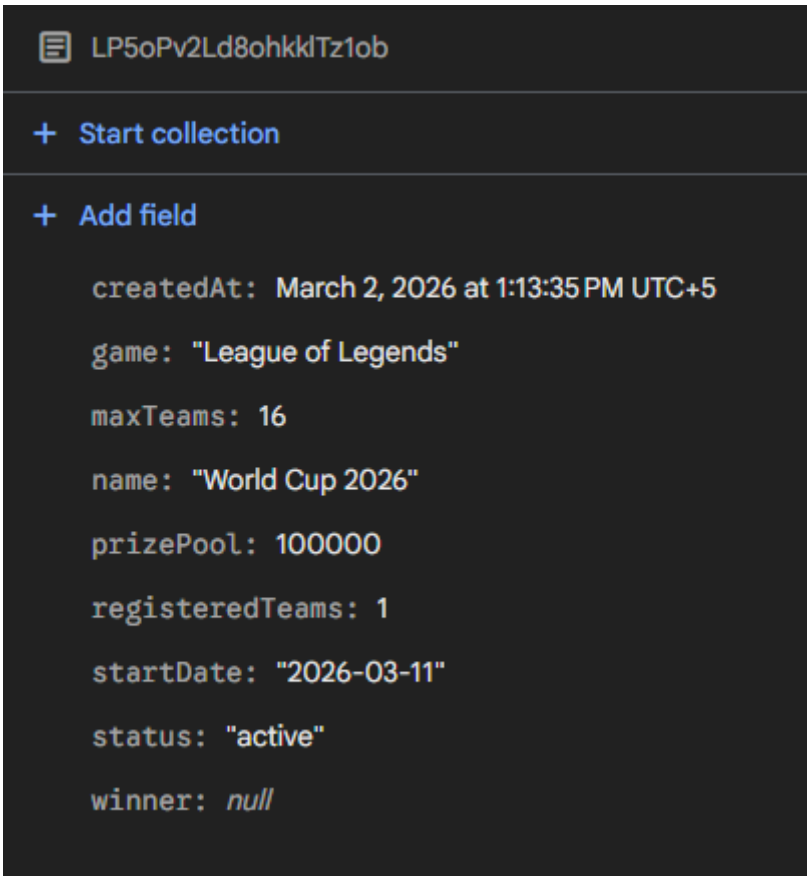

Рисунок 1 - Структура tournaments
Примечание - составлено автором на основании проведенного исследования

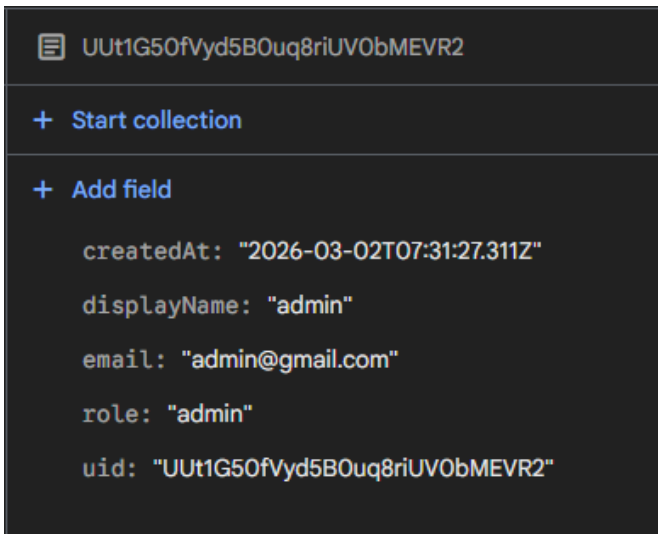

Рисунок 2 - Структура users
Примечание - составлено автором на основании проведенного исследования

1.3 Связи между коллекциями

В проекте используется денормализованная структура данных, характерная для NoSQL-подхода, что обусловлено необходимостью повышения производительности системы и сокращения количества сложных операций объединения данных. Такой подход позволяет оптимизировать работу с базой данных за счёт хранения связанных сущностей в удобном для быстрого доступа виде, даже несмотря на возможное дублирование информации. В условиях высоконагруженных приложений это является оправданным решением, так как снижает задержки при выполнении запросов и повышает общую отзывчивость системы.

Связь Users — Registrations реализована посредством включения поля userId в каждый документ коллекции registrations. Данное решение позволяет эффективно выполнять выборку заявок, принадлежащих конкретному пользователю, без необходимости выполнения дополнительных запросов или объединений данных. Это особенно важно при построении пользовательских интерфейсов, где требуется быстрое отображение персонализированной информации, такой как список поданных заявок, их статусы и связанные данные.

Связь Tournaments — Registrations организована аналогичным образом: каждый документ заявки содержит поле tournamentId, которое указывает на конкретное событие или турнир. Это обеспечивает возможность быстрой фильтрации заявок по определённому турниру, а также упрощает реализацию административных функций, таких как просмотр списка участников, управление заявками и анализ активности. Кроме того, подобная структура данных способствует более гибкой масштабируемости системы и упрощает её адаптацию под изменяющиеся требования.

2. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ (FIRESTORE SCHEMA)

2.1 Выбор темы и описание предметной области

В рамках выполнения учебной практики по модулю ПМ.03 была выбрана тема **«Разработка веб-приложения для организации киберспортивных турниров»**. Актуальность данной темы обусловлена стремительным ростом индустрии электронного спорта и потребностью в автоматизированных инструментах для координации соревновательных процессов.

Анализ предметной области: Предметная область проекта охватывает комплекс информационных процессов, связанных с жизненным циклом киберспортивного мероприятия. Основными функциональными узлами системы являются:

Контент-менеджмент: публикация и актуализация данных о турнирах (название дисциплины, призовой фонд, регламент).

Регистрация участников: сбор и верификация заявок от игроков, контроль лимитов на количество команд.

Разграничение прав доступа: внедрение ролевой модели (User/Admin) для обеспечения информационной безопасности и разделения пользовательского и управленческого интерфейсов.

Обоснование технологического стека: Для реализации серверной части и хранения данных выбрана облачная NoSQL база данных **Cloud Firestore**. Данное решение обосновано следующими техническими преимуществами:

1. **Работа в реальном времени (Real-time):** Архитектура Firestore позволяет транслировать изменения данных (например, обновление счета или списка участников) на клиентские устройства без необходимости принудительной перезагрузки страницы, что критично для динамичной среды киберспорта.
2. **Гибкость структуры документов:** В отличие от реляционных СУБД, NoSQL подход позволяет легко расширять поля документов (например, добавлять новые характеристики для разных игровых дисциплин), не нарушая целостность всей базы.
3. **Масштабируемость и высокая доступность:** Использование облачного решения Google Cloud гарантирует стабильную работу приложения при резком росте количества одновременных запросов в моменты открытия регистраций на крупные турниры.

2.2 Визуальная схема структуры Firestore

В рамках проектирования была разработана архитектура, состоящая из трех ключевых коллекций.

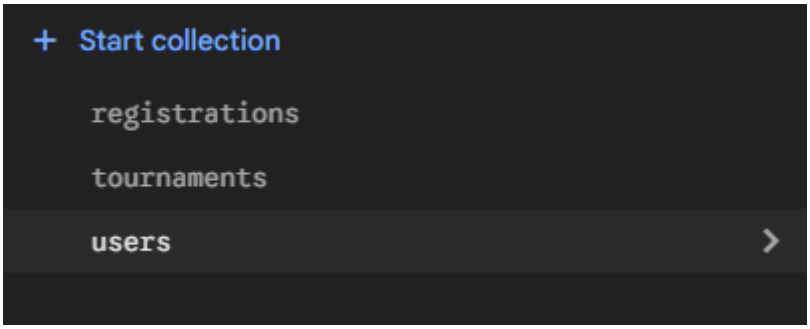

Рисунок 3 - Коллекции

Примечание - составлено автором на основании проведенного исследования

Коллекция tournaments (Турниры): Каждый документ в этой коллекции описывает отдельное событие.
name (string): Название турнира.
game (string): Дисциплина (CS2, Dota 2 и т.д.).
prizePool (number): Сумма призового фонда.
status (string): Состояние турнира (active, completed).
maxTeams (number): Максимальное число команд.
registeredTeams (number): Текущее количество заявок.
description (string): Описание правил и условий.

Коллекция users (Пользователи): Хранит расширенные данные профилей, связанных с Firebase Authentication.
uid (string): Уникальный идентификатор.
displayName (string): Никнейм игрока.
email (string): Электронная почта.
role (string): Роль в системе (user или admin).

Коллекция registrations (Заявки): Служит для фиксации участия пользователей в турнирах.
tournamentId (string): Ссылка на ID турнира.
userId (string): Ссылка на ID пользователя.
teamName (string): Название команды участника.
timestamp (serverTimestamp): Время подачи заявки.

3. ПРОЕКТИРОВАНИЕ САЙТА

3.1 Внешняя структура сайта

Внешняя структура определяет логику перемещения пользователя по разделам приложения. Веб-приложение реализовано по технологии **SPA (Single Page Application)** с использованием библиотеки **Vue Router**. Это позволяет динамически перерисовывать части страницы без полной перезагрузки, что критически важно для производительности и плавности пользовательского опыта.

Навигация построена на системе маршрутов (routes), которые делятся на публичные и защищенные (доступные только авторизованным пользователям или администраторам) Таблица 1 — Внешняя структура сайта (Логика навигации)

	Страница	**Путь (URL)**	**Основное назначение**	**Доступные переходы**
№	**1**	**2**	**3**	**4**
1	**Главная**	/	Лента активных турниров, поиск, фильтрация по играм.	Детальная страница, Профиль, Авторизация.
2	**Детальная**	/tournament/:id	Полная информация о турнире.	Регистрация (участие), Главная.
3	**Личный кабинет**	/profile	История заявок пользователя, изменение игрового имени.	Детальная страница (из истории).
4	**Авторизация**	/login	Форма входа в существующий аккаунт.	Регистрация, Главная.
5	**Регистрация**	/register	Создание нового профиля игрока	Авторизация.

Продолжение таблицы 1

6	**Админ-панель**	/admin	Управление турнирами, пользователями и статистикой.	Любая страница через меню.
Примечание - составлено автором на основании проведенного исследования				

3.2 Внутренняя структура сайта

Внутренняя структура отражает физическое расположение файлов проекта и распределение ответственности между программными модулями. Приложение имеет модульную структуру, что облегчает поддержку и масштабирование кода. **Основные директории проекта:**

/src/components/ — Vue-компоненты (страницы и переиспользуемые элементы интерфейса).

/src/services/ — логика взаимодействия с API Firebase.

/src/assets/ — статические ресурсы (стили, изображения).

Таблица 2 — Внутренняя структура сайта (Назначение программных модулей)

	Файл / Модуль	**Назначение и взаимодействие**
№	**1**	**2**
1	firebase.js	Инициализация Firebase, экспорт auth и db; используется во всех компонентах
2	firebaseService.js	Работа с Firestore (получение и подписка на турниры); Home.vue, Admin.vue
3	router.js	Маршрутизация и проверка ролей; используется в main.js и App.vue
4	App.vue	Корневой компонент с общей структурой интерфейса
5	Home.vue	Главная страница (поиск, фильтры, лента); использует сервис и карточки

Продолжение таблицы 2

6	Admin.vue	CRUD-интерфейс администратора; взаимодействует с сервисом и роутером
7	Profile.vue	Профиль пользователя и заявки; использует авторизацию
Примечание - составлено автором на основании проведенного исследования		

Логика взаимодействия модулей:

Логика взаимодействия модулей в приложении выстроена на основе многоуровневой архитектуры, что позволяет чётко разделить ответственность между различными частями системы и повысить её поддерживаемость. Такой подход обеспечивает гибкость разработки, упрощает масштабирование проекта и делает кодовую базу более структурированной и понятной для дальнейшего развития.

Слой представления представлен компонентами Vue (.vue), которые отвечают за отрисовку пользовательского интерфейса и обработку действий пользователя. Данный уровень взаимодействует с пользователем напрямую, обеспечивая отображение актуальных данных, полученных из сервисного слоя, а также реагирует на события, такие как нажатия кнопок, ввод данных и навигация по приложению. Благодаря реактивной природе Vue, изменения данных автоматически отражаются в интерфейсе, что значительно упрощает разработку динамичных и отзывчивых пользовательских интерфейсов.

Слой сервисов реализован в виде отдельного модуля firebaseService.js, который инкапсулирует всю логику взаимодействия с базой данных Firestore. Это означает, что компоненты не работают с базой данных напрямую, а обращаются к заранее определённым методам, возвращающим уже обработанные и подготовленные данные. Такой подход снижает связанность между слоями, упрощает тестирование и позволяет централизованно управлять всеми операциями чтения и записи данных. Кроме того, это облегчает внесение изменений в структуру данных или используемые API без необходимости модификации компонентов.

Слой безопасности реализован с использованием механизма маршрутизации, настроенного в файле router.js. Перед каждым переходом между страницами выполняется проверка состояния авторизации пользователя с помощью функции onAuthStateChanged, что позволяет определить, имеет ли пользователь доступ к запрашиваемому ресурсу. В случае отсутствия необходимых прав доступ к защищённым маршрутам, таким как административная панель, блокируется, и пользователь перенаправляется на страницу авторизации или главную страницу. Такой механизм обеспечивает базовый уровень защиты приложения и предотвращает несанкционированный доступ к критически важным функциям системы.

Дополнительно следует отметить, что представленная структура файлов проекта (рисунок 4) логически отражает описанную архитектуру и способствует упорядоченной организации кода. В директории src выделены отдельные области ответственности: папка components содержит все пользовательские интерфейсные компоненты, такие как Home.vue, Admin.vue, Profile.vue и другие, отвечающие за различные страницы и элементы интерфейса. Служебные файлы, такие как firebase.js и firebaseService.js, отвечают за

взаимодействие с облачной инфраструктурой, тогда как router.js реализует навигацию и контроль доступа, а main.js выступает точкой входа в приложение. Подобное разделение упрощает навигацию по проекту, делает его более читаемым и поддерживаемым, а также соответствует современным практикам разработки frontend-приложений.

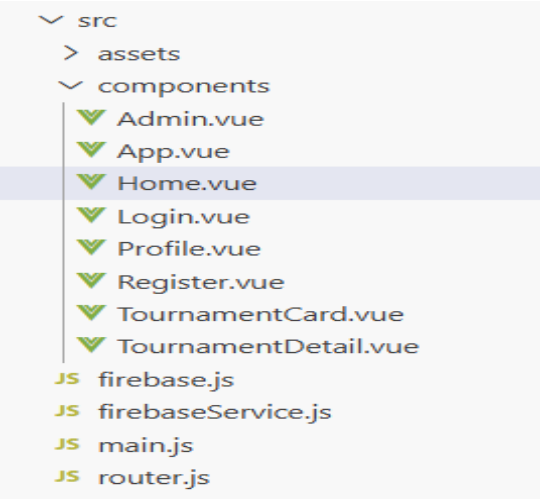

Рисунок 4 - Структура файлов

Примечание - составлено автором на основании проведенного исследования

4. ПРАКТИЧЕСКАЯ ЧАСТЬ

4.1 Настройка Firebase проекта

Работа над проектом «CYBERTOUR» началась с создания облачной инфраструктуры.

Firebase Console: В консоли был создан проект labb-wwork.

Authentication: Включен метод входа по Email и паролю. Это база для идентификации игроков.

Firestore: База данных инициализирована в тестовом режиме.

Связь с кодом: Создан файл firebase.js, где прописан конфиг (apiKey, projectId и т.д.). Этот файл является «входной точкой» для всех сервисов.

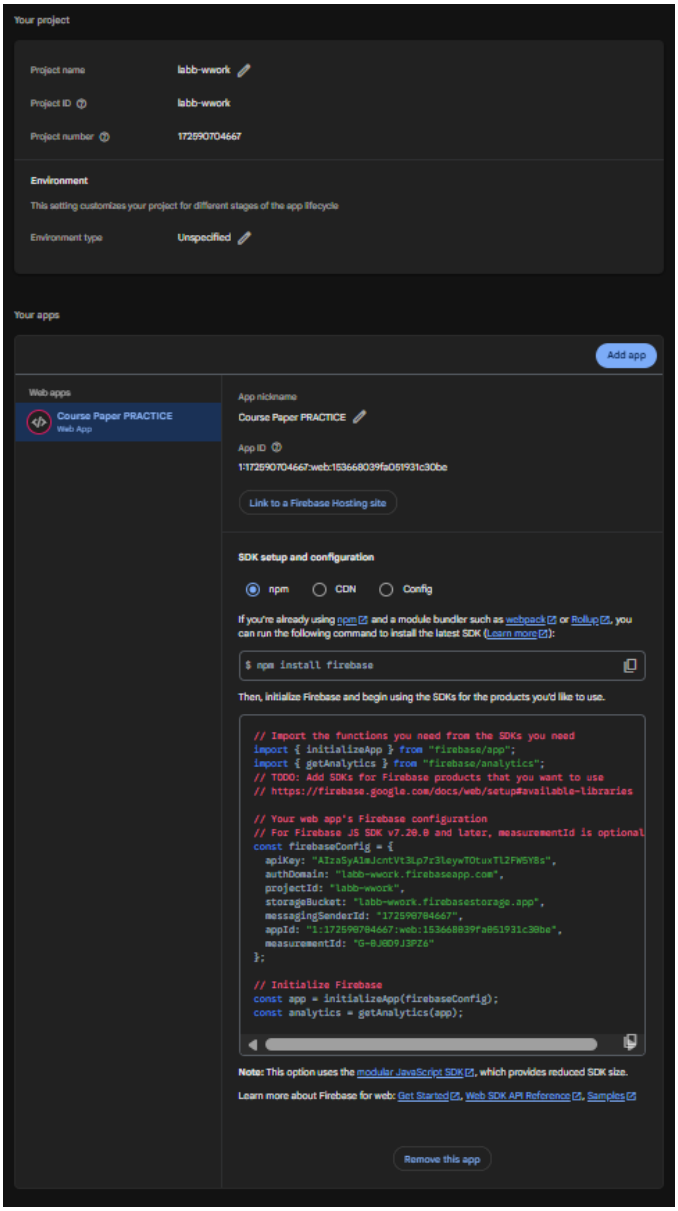

Рисунок 5 - Кодовый конфиг

Примечание - составлено автором на основании проведенного исследования

4.2 Создание схемы базы данных и тестовых данных

На основе спроектированной схемы были созданы коллекции в облаке.

1. **Коллекции:** tournaments (турниры) и users (профили).
2. **Тестовые данные:** Через консоль были добавлены 3-4 турнира (CS2, Dota 2), чтобы проверить, как работает лента на сайте.
3. **Индексы:** Для того чтобы сайт мог одновременно фильтровать турниры по игре и сортировать их по призовому фонду, в Firebase были созданы **составные индексы (Composite Indexes)**. Без них сложные запросы выдавали бы ошибку.

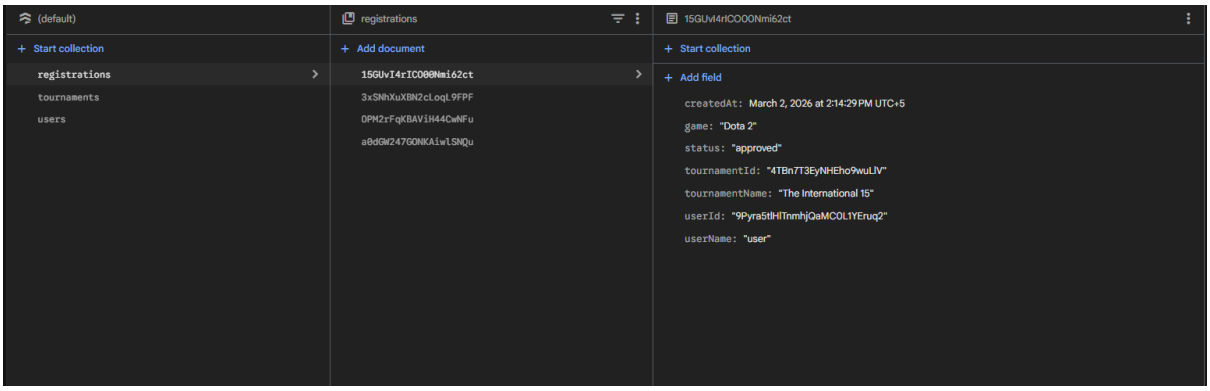

Рисунок 6 - Схема firebase

Примечание - составлено автором на основании проведенного исследования

4.3 Реализация системы аутентификации

Модуль **Register.vue** отвечает за реализацию одного из базовых и наиболее важных процессов в приложении — регистрацию и аутентификацию пользователей. Его функциональность охватывает полный цикл взаимодействия с системой: от создания аккаунта до отслеживания текущего состояния пользователя в приложении. При разработке данного модуля особое внимание было уделено как безопасности, так и удобству использования.

Процесс регистрации реализован с использованием метода createUserWithEmailAndPassword, который предоставляет Firebase Authentication. Данный метод обеспечивает создание учетной записи пользователя с проверкой корректности введённых данных и обработкой возможных ошибок (например, некорректный формат email или слишком простой пароль). Сразу после успешной регистрации дополнительно выполняется запись данных пользователя в базу Firestore: создаётся документ в коллекции users, где, помимо идентификатора пользователя, сохраняется его роль — по умолчанию это значение user. Такой подход позволяет в дальнейшем гибко управлять правами доступа и разграничивать функциональность между обычными пользователями и администраторами.

Для входа в систему используется метод signInWithEmailAndPassword, который обеспечивает проверку учетных данных и аутентификацию пользователя. Этот процесс сопровождается стандартной обработкой ошибок (например, неверный пароль или отсутствие пользователя), что делает взаимодействие с системой более понятным и безопасным для конечного пользователя.

Отдельного внимания заслуживает механизм отслеживания состояния пользователя. С помощью слушателя onAuthStateChanged реализуется постоянный мониторинг статуса аутентификации. Это означает, что приложение «знает», авторизован пользователь или нет, даже после перезагрузки страницы или при повторном открытии сайта. Такой подход избавляет от необходимости повторного входа при каждом обновлении и делает

пользовательский опыт более плавным.

Данный механизм тесно интегрирован с системой маршрутизации приложения. В файле router.js реализованы Navigation Guards, которые используют информацию о текущем состоянии пользователя для защиты определённых страниц. Если пользователь не авторизован или не обладает необходимыми правами, доступ к защищённым маршрутам блокируется, и происходит перенаправление на соответствующую страницу (например, страницу входа или главную).

В совокупности модуль Register.vue формирует надежную основу для управления пользователями, обеспечивая безопасную регистрацию, удобный вход в систему и корректную обработку состояния аутентификации на протяжении всей работы приложения.

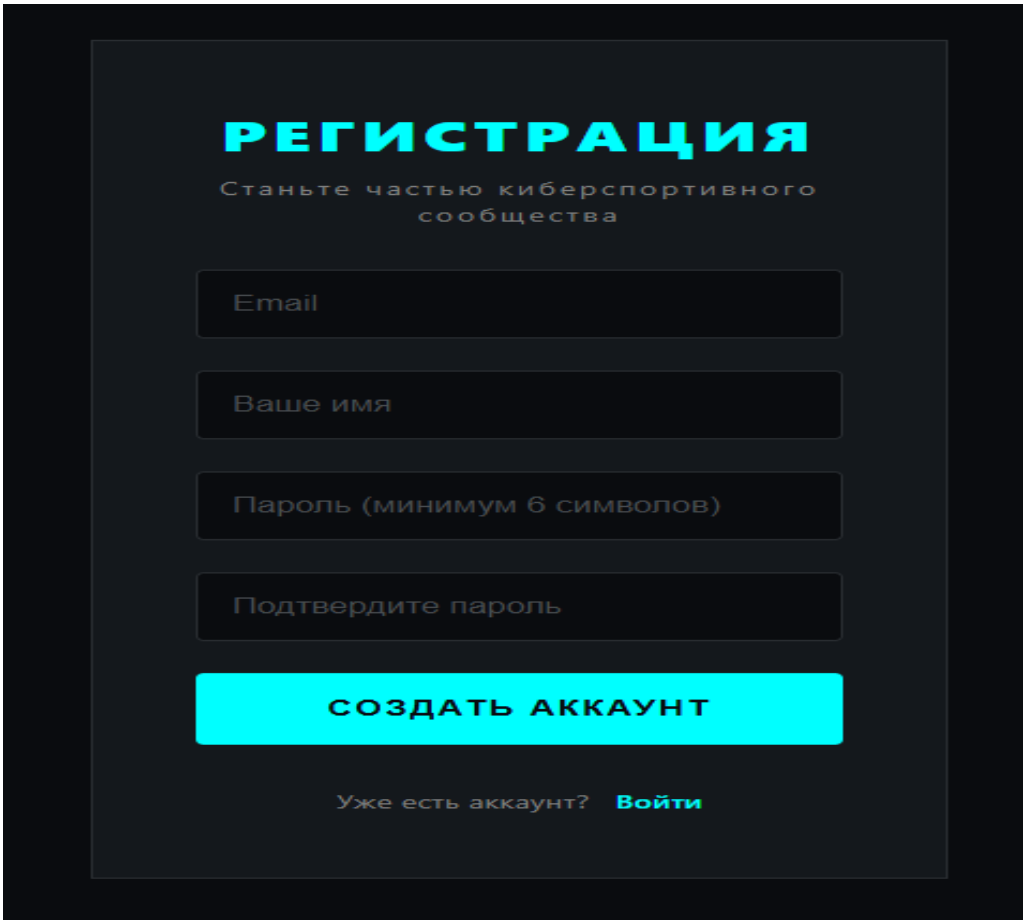

Рисунок 7 - Регистрация на сайте

Примечание - составлено автором на основании проведенного исследования

4.4 Разработка главной страницы с пагинацией

Главная страница приложения (Home.vue) спроектирована как высокопроизводительная «витрина» киберспортивных мероприятий. При разработке данного модуля основной акцент был сделан на обеспечении быстрой первичной отрисовки (**First Contentful Paint**) и минимизации избыточного сетевого трафика между клиентом и облачной базой данных Firestore.

Стратегия оптимизации запросов и лимитирование: В процессе реализации механизмов получения данных было принято архитектурное решение отказаться от единовременной загрузки всего массива документов из коллекции tournaments. Массовая загрузка данных является ресурсозатратной операцией («дорогой» с точки зрения

тарификации Firebase и «медленной» для конечного пользователя).

Техническое решение: Интегрирован метод .limit(15), который жестко ограничивает объем одной порции данных. Это позволяет мгновенно отобразить первые 15 актуальных турниров, значительно снижая нагрузку на оперативную память устройства и ускоряя рендеринг страницы.

Механизм курсорной пагинации: Для обеспечения доступа к полному списку турниров без потери производительности была внедрена система **курсорной пагинации**.

Логика реализации: Функционал базируется на использовании метода .startAfter(), входящего в состав Firebase SDK. Система сохраняет указатель на последний видимый документ текущей выборки. При активации пользователем триггера «Показать еще», приложение инициирует новый асинхронный запрос, передавая сохраненный указатель в качестве точки отсчета.

Результат: Происходит бесшовная подгрузка следующего сегмента данных, который добавляется к существующему списку в реактивном массиве Vue, что исключает дублирование записей и обеспечивает консистентность интерфейса.

Управление состояниями ожидания (Loading States): Учитывая асинхронную природу взаимодействия с облаком, критически важным этапом стала реализация визуальной обратной связи.

Компонент обратной связи: В структуру страницы интегрирован специализированный индикатор — loading-spinner. Данный элемент активируется программно в момент инициации запроса и скрывается только после полного разрешения промиса и обновления данных в DOM.

Значение для UX: Наличие анимированного индикатора подтверждает работоспособность системы в моменты сетевых задержек, предотвращая ощущение «зависания» интерфейса и повышая общее доверие пользователя к программному продукту.

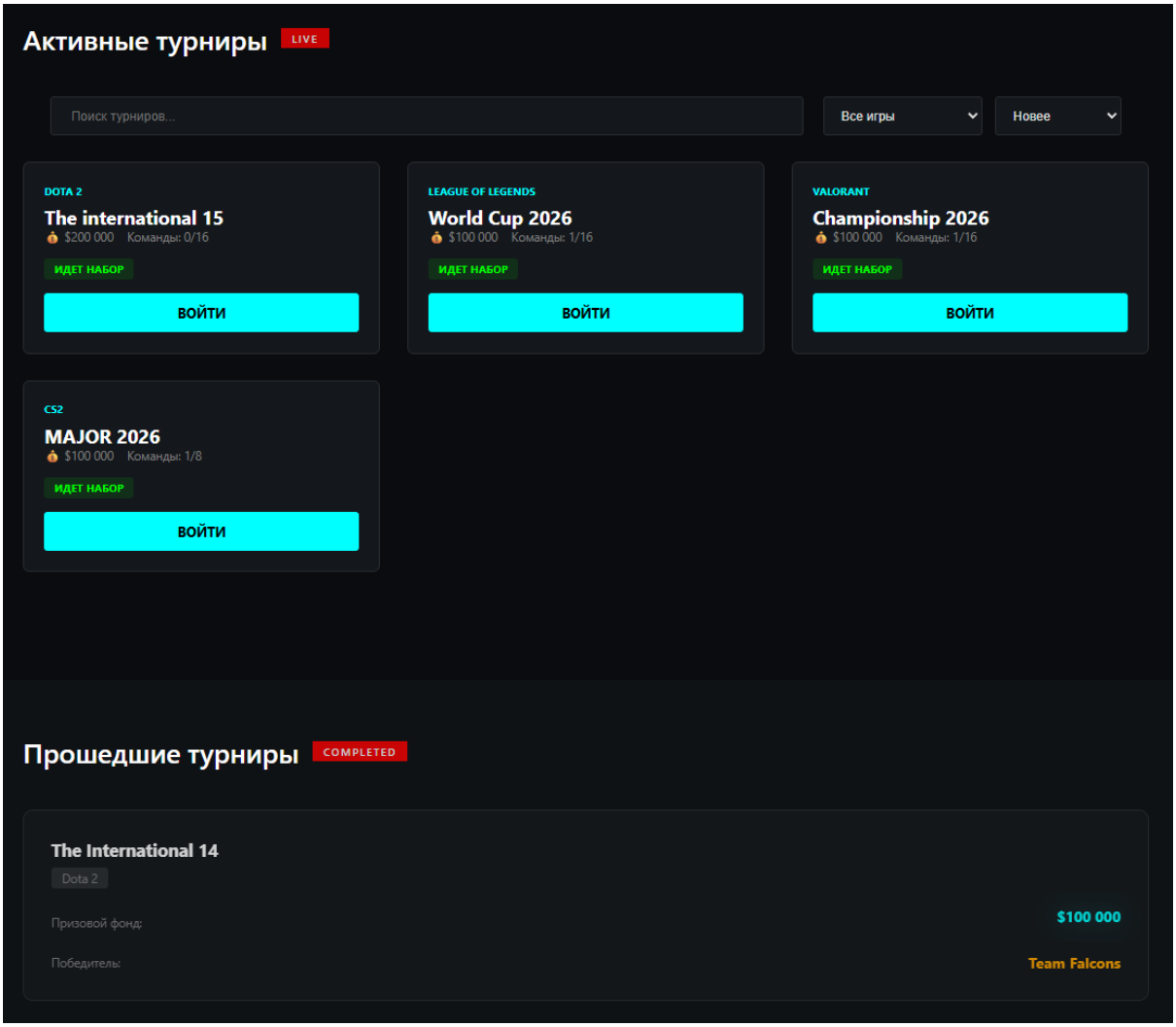

Рисунок 8 - Главная страница

Примечание - составлено автором на основании проведенного исследования

4.5 Реализация механизмов поиска и многоуровневой фильтрации

Для обеспечения высокой скорости навигации и удобства взаимодействия пользователя с обширным каталогом киберспортивных мероприятий, в приложении была разработана гибкая система поиска и фильтрации. Основной задачей данного модуля является минимизация времени, затрачиваемого пользователем на нахождение релевантного контента среди множества активных турниров.

Интеллектуальный поиск по названию: Техническая реализация функции поиска базируется на использовании реактивной связи (двустороннего связывания данных) между текстовым полем ввода и программной переменной searchQuery.

Логика работы: При вводе символов пользователем, система в реальном времени применяет алгоритм фильтрации к массиву данных, хранящемуся в состоянии компонента. Проверка соответствия осуществляется по строковому полю name (название турнира).

Оптимизация: Для повышения отзывчивости интерфейса поиск реализован на стороне клиента с использованием вычисляемых свойств (**Computed Properties**) Vue 3. Это позволяет мгновенно обновлять список карточек на экране без необходимости отправки дополнительных запросов к облачной базе данных, что существенно экономит сетевой трафик и снижает нагрузку на систему.

Серверная фильтрация по игровым дисциплинам: В дополнение к текстовому поиску был внедрен механизм категоризации турниров по конкретным играм (CS2, Dota 2, Valorant и др.).

Интерфейс: Фильтрация реализована через выпадающий список (компонент Select), содержащий перечень поддерживаемых дисциплин.

Технология запросов: В отличие от текстового поиска, выбор категории инициирует выполнение асинхронного запроса к **Firestore** с использованием метода .where('game', '==', selectedGame). Данный подход позволяет получать из облака только строго необходимые документы, что критически важно для масштабируемости проекта.

Интеграция с индексами: Для обеспечения корректной работы этого функционала на стороне Firebase были предварительно настроены правила индексации, что гарантирует высокую скорость выборки данных даже при значительном объеме коллекции.

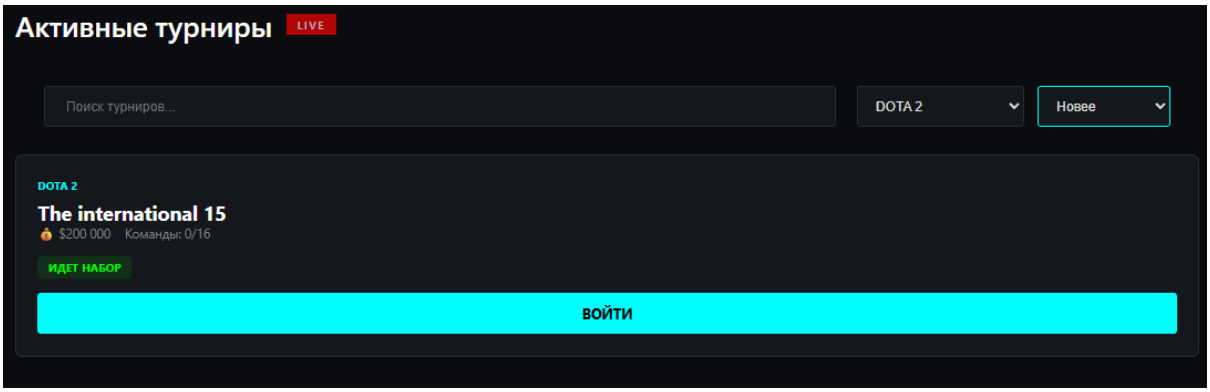

Рисунок 9 - Фильтрация

Примечание - составлено автором на основании проведенного исследования

4.6 Страница детального просмотра (TournamentDetail.vue)

Данный программный модуль выступает в роли критического связующего звена между общим каталогом доступных соревнований и этапом непосредственного участия пользователя в турнирной деятельности. Проектирование страницы детального просмотра потребовало реализации сложной логики обработки динамических параметров и асинхронного взаимодействия с облачным хранилищем.

Механизм динамической маршрутизации: Для обеспечения гибкости навигации была внедрена система **динамической маршрутизации**. Уникальный идентификатор документа (ID) извлекается непосредственно из URL-адреса запроса с помощью хука useRoute(), предоставляемого библиотекой vue-router. Такой подход позволяет приложению в реальном времени определять контекст страницы и формировать точный запрос к конкретному документу в коллекции tournaments. Использование динамических путей (например, /tournament/:id) делает структуру сайта масштабируемой и удобной для индексации.

Асинхронная загрузка данных и управление состоянием: Процесс получения информации реализован через асинхронный метод SDK Firebase — getDoc. Учитывая специфику работы с удаленными серверами, была интегрирована система обработки состояний ожидания:

Состояние загрузки: На время выполнения сетевого запроса пользователю транслируется специализированный графический компонент loading-spinner. Это является важным элементом оптимизации **UX (User Experience)**, так как визуальное подтверждение

процесса загрузки предотвращает преждевременный уход пользователя со страницы и снижает когнитивную нагрузку.

Обработка ошибок: В модуле предусмотрена логика на случай отсутствия документа в базе данных, что гарантирует стабильность работы приложения при переходе по некорректным ссылкам.

Отображение контента и логика представления: После успешного разрешения промиса и получения объекта данных, интерфейс динамически отрисовывает полный набор характеристик турнира:

Метаданные: Дисциплина, подробное текстовое описание регламента, актуальный размер призового фонда и текущий системный статус.

Условный рендеринг: В программный код интегрированы директивы условного отображения. Например, если поле status принимает значение «completed» (завершен), алгоритм автоматически скрывает форму регистрации и выводит блок с информацией о победителе, извлекая данные из соответствующих полей документа.

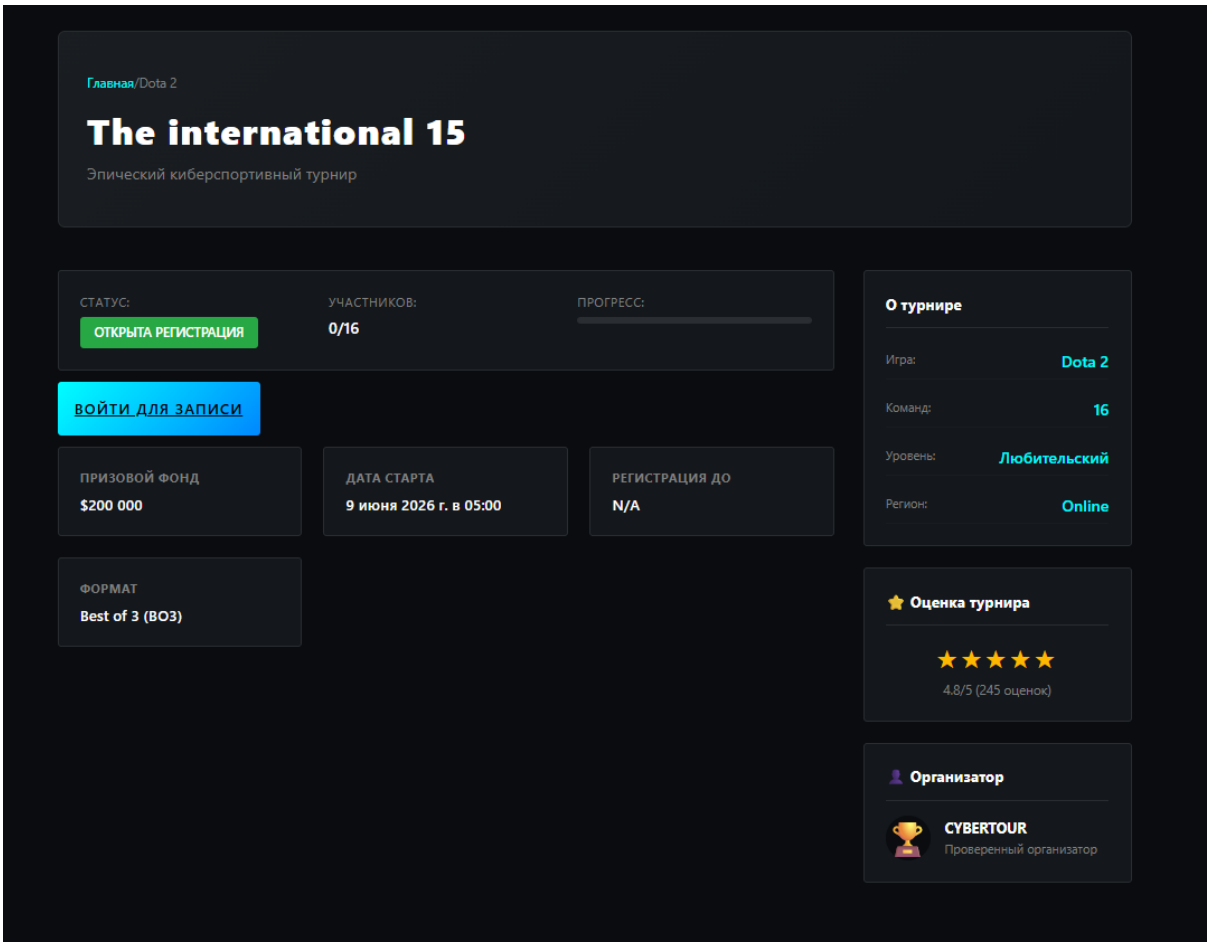

Рисунок 10 - Страница детального просмотра

Примечание - составлено автором на основании проведенного исследования

4.7 Система взаимодействия (Заявки на участие)

Реализация механизма записи на турнир представляет собой один из ключевых интерактивных элементов всего приложения, так как именно через него пользователи напрямую взаимодействуют с системой и становятся участниками соревнований. При разработке данного функционала особое внимание было уделено как удобству пользовательского интерфейса, так и корректности обработки данных на стороне сервера.

На этапе проверки условий реализована логика, обеспечивающая корректность допуска пользователя к участию. Кнопка «Участвовать» становится активной только при выполнении двух обязательных требований: пользователь должен быть авторизован в системе, а в выбранном турнире должны оставаться свободные места. Последнее условие проверяется сравнением значений полей registeredTeams и maxTeams. Такой подход позволяет предотвратить переполнение турнира и исключает возможность участия неавторизованных пользователей, что положительно сказывается на общей надежности системы.

Особое значение имеет обеспечение атомарности операций при создании записи. В момент, когда пользователь нажимает кнопку участия, создаётся новый документ в коллекции registrations, который содержит связку userId и tournamentId. Это позволяет однозначно идентифицировать факт регистрации и в дальнейшем использовать эти данные для аналитики, отображения списка участников или проверки повторной регистрации. Атомарность в данном случае играет важную роль, так как гарантирует, что операция либо выполняется полностью, либо не выполняется вовсе, исключая появление неконсистентных данных.

Дополнительно реализована логика обновления счётчика участников. После успешной регистрации пользователя происходит инкремент значения поля registeredTeams в основном документе турнира. Это необходимо для того, чтобы система всегда отображала актуальное количество занятых мест. Как правило, для таких операций используются специальные механизмы безопасного увеличения значения, что позволяет избежать ошибок при одновременных действиях нескольких пользователей.

В совокупности данный функционал обеспечивает не только удобный и интуитивно понятный процесс записи на турнир, но и поддерживает целостность данных, устойчивость к конкурентным операциям и высокую отзывчивость интерфейса, что является важным требованием для современных веб-приложений.

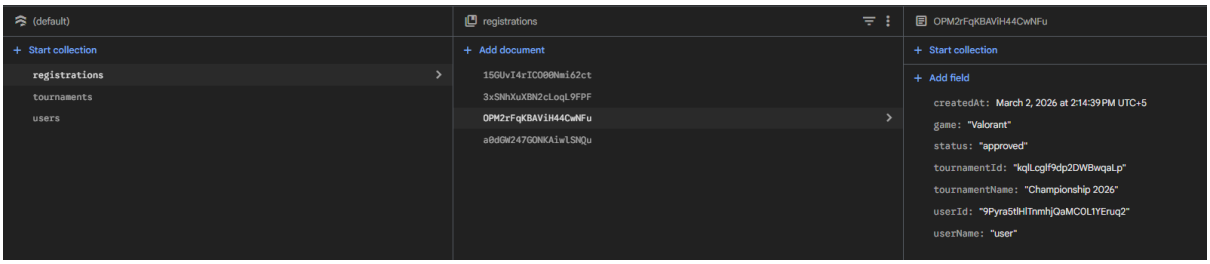

Рисунок 11 - Заявки на участие

Примечание - составлено автором на основании проведенного исследования

4.8 Личный кабинет пользователя (Profile.vue)

Личный кабинет реализован как персональный хаб игрока, объединяющий ключевые функции, связанные с управлением пользовательским профилем и взаимодействием с системой. Он выступает центральной точкой доступа к персонализированной информации, обеспечивая удобную навигацию и быстрый доступ к данным, необходимым пользователю в процессе использования приложения. Такой подход способствует повышению вовлечённости пользователей и улучшению общего пользовательского опыта за счёт концентрации всей важной информации в одном месте.

Отображение данных организовано на основе интеграции с сервисами аутентификации и базы данных. Из Firebase Auth извлекается основная информация о пользователе, в частности его адрес электронной почты, который используется в качестве уникального идентификатора. Дополнительно из Firestore загружаются расширенные пользовательские данные, такие как никнейм, что позволяет формировать более

персонализированный интерфейс. Благодаря этому достигается согласованность данных между различными частями приложения и обеспечивается их актуальность.

Редактирование профиля реализовано через функцию saveName, которая предоставляет пользователю возможность изменять свой игровой ник в удобной и интуитивно понятной форме. При внесении изменений данные мгновенно обновляются в документе пользователя в базе Firestore, что обеспечивает эффект работы в реальном времени. Это позволяет избежать необходимости перезагрузки страницы или повторной авторизации, а также делает взаимодействие с системой более плавным и современным.История активностей пользователя формируется с использованием запросов к базе данных, в частности с применением условия where('userId', '==', uid). Данный механизм позволяет получить список всех турниров, на которые пользователь подавал заявки, и отобразить их в структурированном виде. Это даёт игроку возможность отслеживать свою активность, контролировать участие в предстоящих событиях и анализировать собственную вовлечённость в игровую среду. Кроме того, подобный функционал может служить основой для дальнейшего расширения системы, например, добавления статистики или рекомендаций.

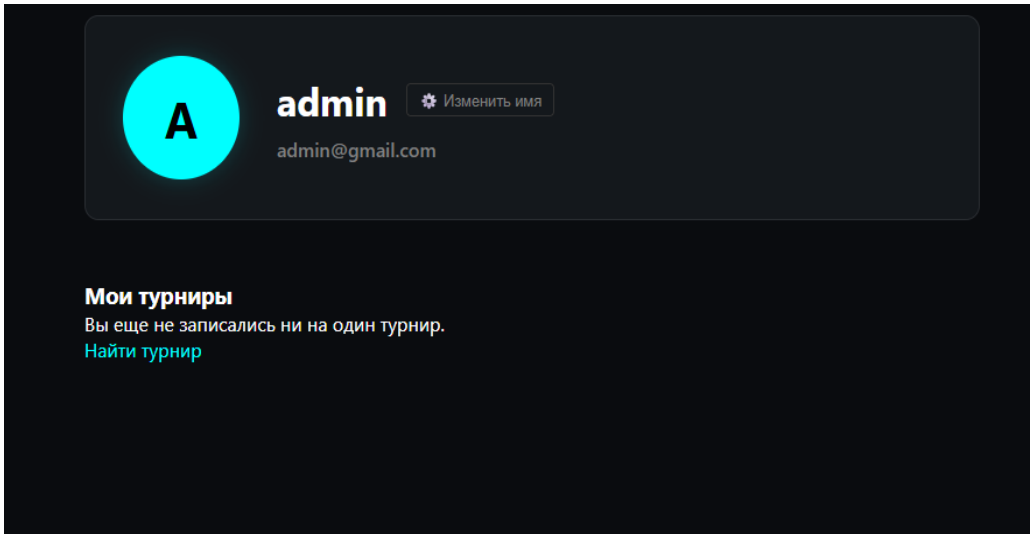

Рисунок 12 - Личный кабинет пользователя

Примечание - составлено автором на основании проведенного исследования

4.9 Реализация режима реального времени (Real-time)

Для того чтобы веб-приложение выглядело более «живым», динамичным и отзывчивым к действиям пользователей, в ряде ключевых модулей был применён механизм подписки на изменения данных с использованием слушателя onSnapshot(). Это решение позволило существенно повысить интерактивность интерфейса и приблизить пользовательский опыт к работе с приложениями в реальном времени.

Принцип работы данного подхода основан на использовании возможностей базы данных Firestore, которая поддерживает реактивную модель взаимодействия. Как только в базе данных происходят какие-либо изменения — например, администратор обновляет статус турнира, добавляет новую запись или изменяется количество участников — сервер Firebase автоматически и практически мгновенно «проталкивает» обновлённые данные на клиентскую сторону. При этом нет необходимости вручную инициировать повторный запрос к серверу: обновление происходит в фоновом режиме через установленное соединение.

Такой механизм позволяет реализовать постоянную синхронизацию между сервером и клиентом, обеспечивая актуальность отображаемой информации в любой момент времени.

Это особенно важно для сценариев, где данные могут часто изменяться и имеют значение для пользователя «здесь и сейчас».

В результате использования onSnapshot() достигается значительное улучшение пользовательского опыта. Пользователю больше не требуется обновлять страницу или выполнять дополнительные действия для получения свежих данных — информация, такая как количество свободных мест, список участников или текущий статус соревнования, обновляется автоматически. Это делает взаимодействие с системой более плавным, удобным и современным, а также повышает общее восприятие качества приложения.

4.10 Админ-панель (Admin.vue)

Админ-панель представляет собой наиболее технически сложный и функционально насыщенный модуль системы, который наглядно демонстрирует практическое применение операций CRUD (Create, Read, Update, Delete). Реализация данного компонента потребовала комплексного подхода к проектированию интерфейса, логики взаимодействия с базой данных и механизмов разграничения прав доступа.

С точки зрения безопасности доступ к административной части строго ограничен. Для этого используются Navigation Guards, настроенные в файле router.js, которые выполняют проверку прав пользователя при попытке перехода на защищённые маршруты. В случае, если в базе данных значение поля role у пользователя не соответствует значению admin, система автоматически выполняет перенаправление на главную страницу. Такой механизм позволяет эффективно предотвратить несанкционированный доступ к административным функциям и обеспечивает дополнительный уровень защиты приложения.

Функционал управления контентом реализован с акцентом на удобство и гибкость. В административной панели предусмотрена форма для добавления новых турниров, работающая через метод addDoc, что позволяет оперативно вносить новые данные в систему. Помимо этого, реализованы возможности редактирования и удаления уже существующих турниров, что даёт администратору полный контроль над актуальностью и корректностью размещённой информации. Такой подход обеспечивает централизованное управление контентом и упрощает процесс его обновления.

Отдельного внимания заслуживает модуль статистики, который предоставляет администратору сводную информацию о состоянии системы. С помощью функции getAdminStats осуществляется вычисление общего количества зарегистрированных пользователей, а также числа активных турниров. Это позволяет быстро оценить текущую активность платформы и принять при необходимости управленческие решения на основе актуальных данных.

В совокупности админ-панель выступает ключевым инструментом администрирования системы, объединяя в себе расширенные возможности управления данными, строгий контроль доступа и инструменты аналитики, что делает её важной частью всей архитектуры приложения.

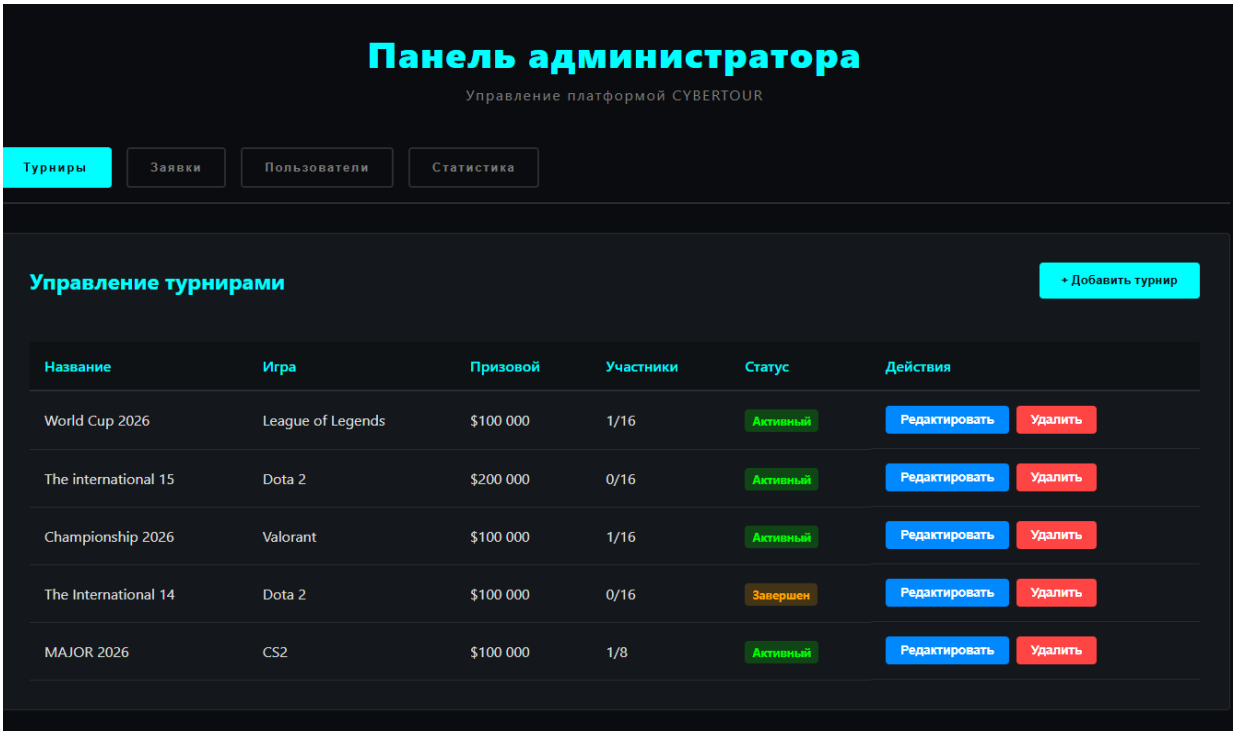

Рисунок 13 - Панель администратора

Примечание - составлено автором на основании проведенного исследования

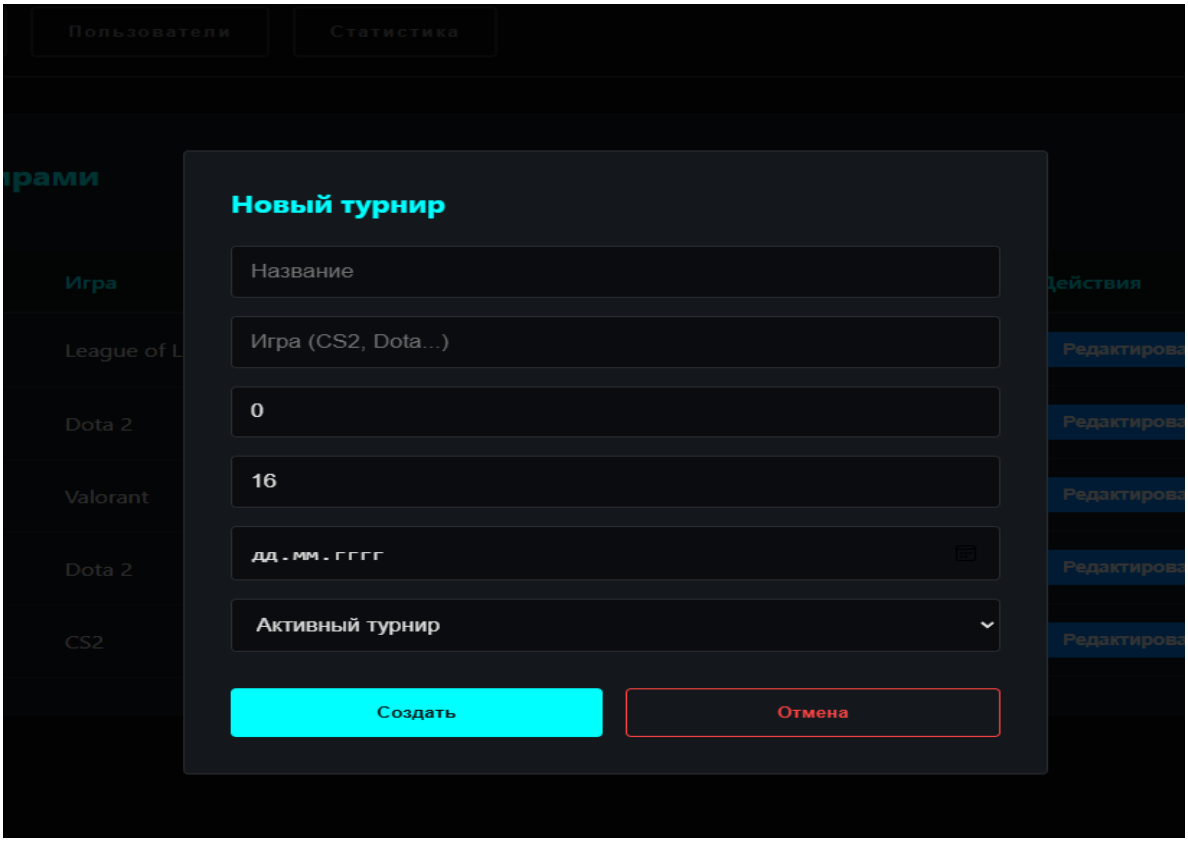

Рисунок 14 - Форма создания турнира

Примечание - составлено автором на основании проведенного исследования

4.11 Оптимизация и правила безопасности (Firestore Rules)

Для обеспечения надежной защиты проекта от несанкционированного доступа были тщательно разработаны и внедрены соответствующие правила безопасности, направленные на разграничение прав пользователей и предотвращение потенциальных угроз.

В рамках настройки системы безопасности предусмотрен публичный уровень доступа, при котором операция чтения данных о турнирах остается доступной для всех пользователей без исключения. Это позволяет обеспечить открытость информации и удобство ознакомления с доступными турнирами без необходимости прохождения процедуры аутентификации.

В то же время для выполнения более чувствительных действий, таких как создание заявок на участие, применяется ограничение приватного доступа. Данная операция разрешена исключительно авторизованным пользователям, что реализуется через проверку условия request.auth != null. Это гарантирует, что только зарегистрированные и прошедшие аутентификацию пользователи могут взаимодействовать с системой на более глубоком уровне.

Особое внимание уделено административным функциям, связанным с управлением турнирами. Операции создания и удаления турниров доступны только пользователям с соответствующими правами администратора. Для этого дополнительно реализована проверка роли пользователя, которая хранится в его документе. Такой подход обеспечивает дополнительный уровень контроля и предотвращает возможность выполнения критически важных действий пользователями без соответствующих полномочий.

В совокупности данные меры формируют комплексную систему безопасности, которая сочетает в себе открытость базовой информации и строгий контроль над операциями, требующими повышенного уровня доверия.

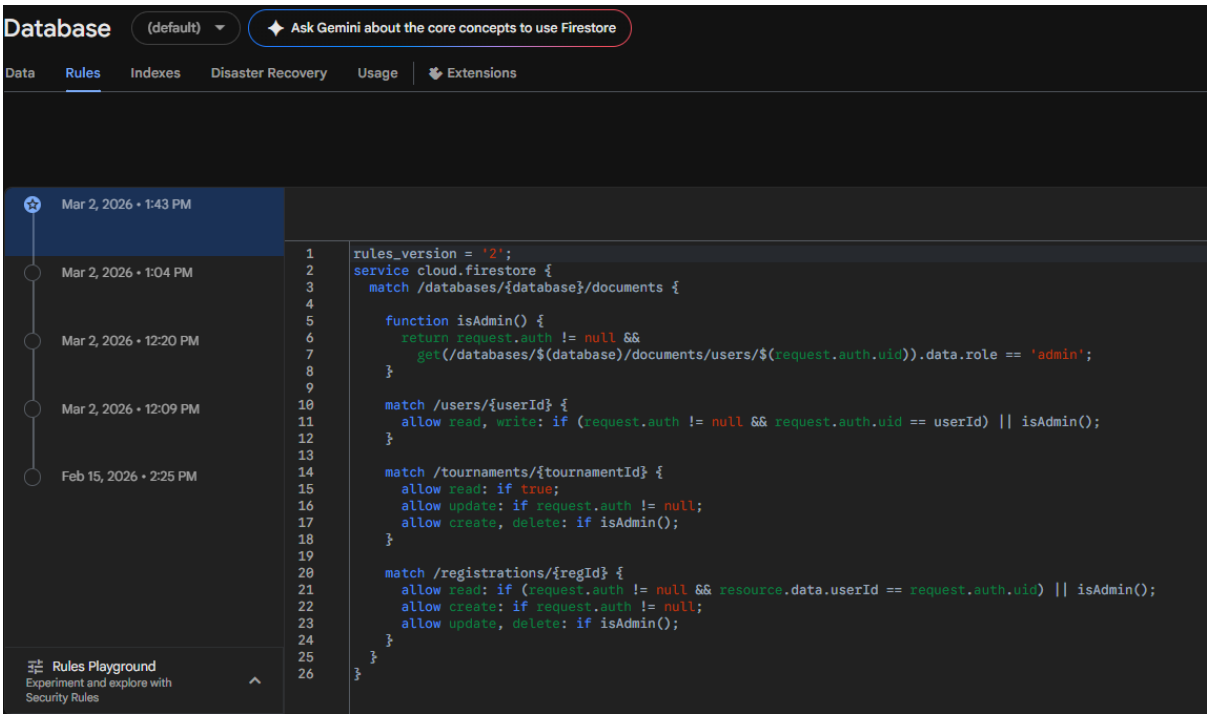

Рисунок 15 – Firestore rules

Примечание - составлено автором на основании проведенного исследования

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики по профессиональному модулю ПМ.03 я реализовал полноценный цикл разработки веб-приложения «CYBERTOUR». Данная работа позволила мне не только закрепить теоретические знания по фронтенд-разработке, но и освоить современные облачные технологии на базе платформы Firebase.

В процессе выполнения проекта мною были достигнуты следующие результаты:

Спроектирована и внедрена база данных: Я разработал NoSQL-структуру в Cloud Firestore, которая эффективно связывает данные о турнирах, профили игроков и их заявки на участие. Это научило меня работать с денормализованными данными и настраивать составные индексы для сложных поисковых запросов.

Реализована серверная логика без бэкенда: С помощью Firebase SDK я настроил систему аутентификации пользователей и управление состоянием сессии. Я применил методы onAuthStateChanged и Navigation Guards в роутинге, что позволило мне надежно защитить личные кабинеты и панель администратора.

Разработан интерактивный интерфейс: Используя Vue 3 (Composition API), я создал динамическую ленту турниров с пагинацией и фильтрацией в реальном времени. Я уделил особое внимание UX, добавив индикаторы загрузки и мгновенное обновление данных через слушатели onSnapshot.

Обеспечена безопасность данных: Я самостоятельно написал и протестировал правила безопасности (Firestore Rules), которые разграничивают права доступа. Это гарантирует, что обычный пользователь не может удалять чужие данные или получать доступ к функциям администратора.

Личные выводы: За время практики я приобрел важные профессиональные компетенции в области Fullstack-разработки. Самым сложным и интересным этапом для меня стала интеграция Vue-компонентов с облачными сервисами и работа с правами доступа. Я убедился, что использование BaaS-решений (Backend as a Service) значительно ускоряет разработку и позволяет сфокусироваться на качестве интерфейса.

Я считаю, что поставленные цели практики выполнены мною в полном объеме. Полученный опыт и готовый проект станут хорошей базой для моей дальнейшей профессиональной деятельности в сфере программной инженерии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. **Google Firebase Documentation.** Cloud Firestore: основы и проектирование структуры данных [Электронный ресурс]. - Режим доступа: https://firebase.google.com/docs/firestore
2. **Vue.js Official Guide.** Документация прогрессивного фреймворка Vue 3 (Composition API) [Электронный ресурс]. - Режим доступа: https://vuejs.org/guide/
3. **Vue Router Documentation.** Организация динамической навигации в SPA-приложениях [Электронный ресурс]. - Режим доступа: https://router.vuejs.org/
4. МР-1 Методические рекомендации по оформлению письменных работ. - Алматы: ТОО «Колледж Хекслет», 2024. - 24 с.
5. Флэнаган, Д. JavaScript. Подробное руководство. / Д. Флэнаган. - 7-е изд. - СПб.: Символ-Плюс, 2021. – 720 с.
6. Никсон, Р. Создаем динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5. / Р. Никсон. - 5-е изд. - СПб.: Питер, 2019. - 816 с
7. Чейз, С. Vue.js в действии. / С. Чейз. - М.: ДМК Пресс, 2019. – 302 с.
8. Кантор, И. Современный учебник JavaScript (javascript.info) [Электронный ресурс]. - Режим доступа: https://javascript.info/
9. SkyPro. Бесплатные ресурсы и документация для изучения программирования [Электронный ресурс]. - Режим доступа: https://sky.pro/wiki/profession/besplatnye-resursy-dlya-obucheniya-programmirovaniyu/
10. MDN Web Docs. Документация по HTML, CSS и JavaScript [Электронный ресурс]. - Режим доступа: https://developer.mozilla.org/

ПРИЛОЖЕНИЯ

Приложение 1 – firebase.js

```
1   import { initializeApp } from "firebase/app";
2   import { getAuth } from "firebase/auth";
3   import { getFirestore } from "firebase/firestore";
4   import { getStorage } from "firebase/storage";
5
6   const firebaseConfig = {
7     apiKey: "AIzaSyA1mJcntVt3Lp7r3leywTOtuxTl2FW5Y8s",
8     authDomain: "labb-wwork.firebaseapp.com",
9     projectId: "labb-wwork",
10    storageBucket: "labb-wwork.firebasestorage.app",
11    messagingSenderId: "172590704667",
12    appId: "1:172590704667:web:153668039fa051931c30be",
13    measurementId: "G-0J0D9J3PZ6"
14  };
15
16  const app = initializeApp(firebaseConfig);
17  const auth = getAuth(app);
18  const db = getFirestore(app);
19  const storage = getStorage(app);
20
21  export { auth, db, storage };
```

```
1
2   import {
3     collection,
4     query,
5     where,
6     orderBy,
7     limit,
8     startAfter,
9     getDocs,
10    getDoc,
11    doc,
12    onSnapshot,
13    writeBatch
14  } from 'firebase/firestore';
15  import { db } from './firebase';
16
17  /**
18   * @param {string} sortBy - сортировка ('newest', 'prize-high', 'prize-low')
19   * @param {string} game - фильтр по игре (опционально)
20   * @param {number} pageSize - размер страницы
21   * @returns {Promise<Array>} массив турниров
22   */
23  export const getTournamentsOptimized = async (sortBy = 'newest', game = null, pageSize = 15) => {
24    try {
25      let q;
26
27      const orderByField = sortBy === 'newest' ? 'createdAt' : 'prizePool';
28      const direction = sortBy === 'prize-low' ? 'asc' : 'desc';
29
30      if (game) {
31        q = query(
32          collection(db, 'tournaments'),
33          where('game', '==', game),
34          orderBy(orderByField, direction),
35          limit(pageSize)
36        );
37      } else {
38        q = query(
39          collection(db, 'tournaments'),
40          orderBy(orderByField, direction),
41          limit(pageSize)
42        );
43      }
44
45      const snapshot = await getDocs(q);
46      return snapshot.docs.map(doc => ({
47        id: doc.id,
48        ...doc.data()
49      }));
50    } catch (error) {
51      console.error('❌ Ошибка загрузки турниров:', error);
52      return [];
53    }
54  };
55
56  /**
57   * @param {Function} callback - функция, которая вызывается при изменении
58   * @param {number} pageSize - размер страницы
59   * @returns {Function} функция для отписки
60   */
61  export const subscribeToTournaments = (callback, pageSize = 15) => {
62    const q = query(
63      collection(db, 'tournaments'),
64      orderBy('createdAt', 'desc'),
65      limit(pageSize)
66    );
67
68    return onSnapshot(
69      q,
70      (snapshot) => {
71        const tournaments = snapshot.docs.map(doc => ({
72          id: doc.id,
73          ...doc.data()
74        }));
75        callback(tournaments);
76      },
77      (error) => {
78        console.error('❌ Ошибка подписки на турниры:', error);
79      }
80    );
81  };
```

```
83   /**
84    * @param {string} tournamentId - ID турнира
85    * @returns {Promise<Object|null>} данные турнира или null
86    */
87   export const getTournamentById = async (tournamentId) => {
88     try {
89       const docRef = doc(db, 'tournaments', tournamentId);
90       const docSnap = await getDoc(docRef);
91
92       if (docSnap.exists()) {
93         return { id: docSnap.id, ...docSnap.data() };
94       }
95       return null;
96     } catch (error) {
97       console.error('❌ Ошибка загрузки турнира:', error);
98       return null;
99     }
100  };
101
102  /**
103   * @param {string} userId - ID пользователя
104   * @returns {Promise<Array>} массив регистраций
105   */
106  export const getUserRegistrations = async (userId) => {
107    try {
108      const q = query(
109        collection(db, 'registrations'),
110        where('userId', '==', userId),
111        orderBy('createdAt', 'desc'),
112        limit(50)
113      );
114
115      const snapshot = await getDocs(q);
116      return snapshot.docs.map(doc => ({
117        id: doc.id,
118        ...doc.data()
119      }));
120    } catch (error) {
121      console.error('Ошибка загрузки регистраций:', error);
122      return [];
123    }
124  };
125
126  /**
127   * @param {string} userId - ID пользователя
128   * @param {Function} callback - функция для обновлений
129   * @returns {Function} функция для отписки
130   */
131  export const subscribeToUserRegistrations = (userId, callback) => {
132    const q = query(
133      collection(db, 'registrations'),
134      where('userId', '==', userId),
135      orderBy('createdAt', 'desc')
136    );
137
138    return onSnapshot(
139      q,
140      (snapshot) => {
141        const registrations = snapshot.docs.map(doc => ({
142          id: doc.id,
143          ...doc.data()
144        }));
145        callback(registrations);
146      },
147      (error) => {
148        console.error('Ошибка подписки на регистрации:', error);
149      }
150    );
151  };
152
153  /**
154   * @param {string} tournamentId - ID турнира
155   * @returns {Promise<Array>} массив регистраций
156   */
157  export const getTournamentRegistrations = async (tournamentId) => {
158    try {
159      const q = query(
160        collection(db, 'registrations'),
161        where('tournamentId', '==', tournamentId),
162        orderBy('createdAt', 'desc'),
163        limit(100)
164      );
```

```
157  export const getTournamentRegistrations = async (tournamentId) => {
158    try {
159      const q = query(
160        collection(db, 'registrations'),
161        where('tournamentId', '==', tournamentId),
162        orderBy('createdAt', 'desc'),
163        limit(100)
164      );
165
166      const snapshot = await getDocs(q);
167      return snapshot.docs.map(doc => ({
168        id: doc.id,
169        ...doc.data()
170      }));
171    } catch (error) {
172      console.error('❌ Ошибка загрузки регистраций турнира:', error);
173      return [];
174    }
175  };
176
177  /**
178   * @param {string} tournamentId - ID турнира
179   * @returns {Promise<Array>} массив отзывов
180   */
181  export const getTournamentReviews = async (tournamentId) => {
182    try {
183      const q = query(
184        collection(db, 'reviews'),
185        where('tournamentId', '==', tournamentId),
186        orderBy('createdAt', 'desc'),
187        limit(20)
188      );
189
190      const snapshot = await getDocs(q);
191      return snapshot.docs.map(doc => ({
192        id: doc.id,
193        ...doc.data()
194      }));
195    } catch (error) {
196      console.error('❌ Ошибка загрузки отзывов:', error);
197      return [];
198    }
199  };
200
201  /**
202   * @param {string} tournamentId - ID турнира
203   * @param {Function} callback - функция для обновлений
204   * @returns {Function} функция для отписки
205   */
206  export const subscribeToTournamentReviews = (tournamentId, callback) => {
207    const q = query(
208      collection(db, 'reviews'),
209      where('tournamentId', '==', tournamentId),
210      orderBy('createdAt', 'desc'),
211      limit(20)
212    );
213
214    return onSnapshot(
215      q,
216      (snapshot) => {
217        const reviews = snapshot.docs.map(doc => ({
218          id: doc.id,
219          ...doc.data()
220        }));
221        callback(reviews);
222      },
223      (error) => {
224        console.error('❌ Ошибка подписки на отзывы:', error);
225      }
226    );
227  };
228
229  /**
230   * @param {string} game - название игры
231   * @param {string} excludeId - ID турнира для исключения
232   * @param {number} limit_num - количество результатов
233   * @returns {Promise<Array>} массив похожих турниров
234   */
235  export const getRelatedTournaments = async (game, excludeId = null, limit_num = 3) => {
236    try {
```

```
export const batchUpdateTournaments = async (updates) => {
  try {
    const batch = writeBatch(db);

    updates.forEach(({ doc: docId, data }) => {
      const docRef = doc(db, 'tournaments', docId);
      batch.update(docRef, data);
    });

    await batch.commit();
    console.log('Пакетное обновление завершено');
  } catch (error) {
    console.error('Ошибка пакетного обновления:', error);
  }
};

/**
 * @returns {Promise<Object>} статистика
 */
export const getAdminStats = async () => {
  try {
    const tournamentsSnap = await getDocs(collection(db, 'tournaments'));
    const tournamentCount = tournamentsSnap.size;

    const registrationsSnap = await getDocs(collection(db, 'registrations'));
    const registrationCount = registrationsSnap.size;

    const usersSnap = await getDocs(collection(db, 'users'));
    const userCount = usersSnap.size;

    return {
      tournaments: tournamentCount,
      registrations: registrationCount,
      users: userCount,
      timestamp: new Date().toISOString()
    };
  } catch (error) {
    console.error('Ошибка загрузки статистики:', error);
    return null;
  }
};

export default {
  getTournamentsOptimized,
  subscribeToTournaments,
  getTournamentById,
  getUserRegistrations,
  subscribeToUserRegistrations,
  getTournamentRegistrations,
  getTournamentReviews,
  subscribeToTournamentReviews,
  getRelatedTournaments,
  batchUpdateTournaments,
  getAdminStats
};
```

Приложение 3. router.js

```
1   import { createRouter, createWebHistory } from "vue-router";
2   import { auth } from "./firebase";
3   import { doc, getDoc } from "firebase/firestore";
4   import { db } from "./firebase";
5   import { onAuthStateChanged } from "firebase/auth";
6   import Home from "./components/Home.vue";
7   import Login from "./components/Login.vue";
8   import Register from "./components/Register.vue";
9   import Profile from "./components/Profile.vue";
10  import Admin from "./components/Admin.vue";
11  import TournamentDetail from "./components/TournamentDetail.vue";
12
13  const routes = [
14    { path: "/", component: Home, props: true },
15    { path: "/login", component: Log import Register
16    { path: "/register", component: Register },
17    { path: "/tournament/:id", component: TournamentDetail, props: true },
18    { path: "/profile", component: Profile, meta: { requiresAuth: true }, props: true },
19    { path: "/admin", component: Admin, meta: { requiresAdmin: true }, props: true },
20  ];
21
22  const router = createRouter({
23    history: createWebHistory(),
24    routes,
25  });
26
27  router.beforeEach(async (to, from, next) => {
28    const requiresAuth = to.matched.some(record => record.meta.requiresAuth);
29    const requiresAdmin = to.matched.some(record => record.meta.requiresAdmin);
30
31    if (requiresAuth || requiresAdmin) {
32      await new Promise((resolve) => {
33        const unsubscribe = onAuthStateChanged(auth, (user) => {
34          unsubscribe();
35          resolve(user);
36        });
37      });
38
39      if (auth.currentUser) {
40        if (requiresAdmin) {
41          try {
42            const userDoc = await getDoc(doc(db, "users", auth.currentUser.uid));
43            if (userDoc.exists() && userDoc.data().role === 'admin') {
44              next();
45            } else {
46              alert('Доступ запрещен. Требуются права администратора.');
47              next('/');
48            }
49          } catch (error) {
50            console.error('Ошибка проверки прав:', error);
51            next('/');
52          }
53        } else {
54          next();
55        }
56      } else {
57        next('/login');
58      }
59    } else {
60      next();
61    }
62  });
63
64  export default router;
```

Приложение 4. Home.vue

```
1   <template>
2     <div class="home-container">
3       <div class="hero-section">
4         <div class="hero-content">
5           <h1 class="hero-title">CYBERTOUR</h1>
6           <p class="hero-subtitle">Платформа для киберспортивных турниров</p>
7
8           <div v-if="!isAuthenticated" class="hero-buttons">
9             <router-link to="/register" class="btn btn-primary">ПРИСОЕДИНИТЬСЯ</router-link>
10          </div>
11          <div v-else class="welcome-message">
12            <h2>С возвращением, {{ userName || userEmail }}!</h2>
13            <p>Готовы к новым победам?</p>
14          </div>
15        </div>
16      </div>
17
18      <div class="tournaments-section">
19        <div class="section-header">
20          <h2 class="section-title">Активные турниры</h2>
21          <span class="section-badge">LIVE</span>
22        </div>
23
24        <!-- Поиск и фильтры -->
25        <div class="filters-container">
26          <div class="search-box">
27            <input
28              v-model="searchQuery"
29              type="text"
30              placeholder="Поиск турниров..."
31              class="search-input"
32            />
33          </div>
34          <div class="filter-controls">
35            <select v-model="selectedGame" class="filter-select">
36              <option value="">Все игры</option>
37              <option value="CS2">CS2</option>
38              <option value="Dota 2">DOTA 2</option>
39              <option value="Valorant">VALORANT</option>
40              <option value="League of Legends">League of Legends</option>
41            </select>
42            <select v-model="sortBy" class="filter-select">
43              <option value="newest">Новее</option>
44              <option value="prize-high">Призовой ↓</option>
45              <option value="prize-low">Призовой ↑</option>
46            </select>
47          </div>
48        </div>
49
50        <div class="tournaments-grid">
51          <TournamentCard
52            v-for="tournament in displayedTournaments"
53            :key="tournament.id"
54            :tournament="tournament"
55            :is-authenticated="isAuthenticated"
56            :user-id="userId"
57            :user-name="userName"
58            @registered="loadTournaments"
59          />
60        </div>
61
62        <div v-if="displayedTournaments.length === 0" class="empty-state">
63          <p>Турниры не найдены. Попробуйте изменить фильтры.</p>
64        </div>
65
66        <div v-if="loading" class="loading">Загрузка турниров...</div>
67
68        <div v-if="hasMoreTournaments" class="load-more-container">
69          <button @click="loadMore" class="btn-load-more">
70            Показать еще турниры
71          </button>
72        </div>
73      </div>
74
75      <div class="past-tournaments-section">
76        <div class="section-header">
77          <h2 class="section-title">Прошедшие турниры</h2>
78          <span class="section-badge">COMPLETED</span>
79        </div>
80
81        <div class="tournaments-grid">
```

```
80
81        <div class="tournaments-grid">
82          <div v-for="tournament in pastTournaments" :key="tournament.id" class="tournament-card past-card">
83            <div class="tournament-header">
84              <h3 class="tournament-name">{{ tournament.name }}</h3>
85              <span class="game-tag">{{ tournament.game }}</span>
86            </div>
87            <div class="tournament-info">
88              <div class="prize-info">
89                <span class="prize-label">Призовой фонд:</span>
90                <span class="prize-amount">${{ tournament.prizePool.toLocaleString() }}</span>
91              </div>
92              <div class="winner-info">
93                <span class="winner-label">Победитель:</span>
94                <span class="winner-name">{{ tournament.winner }}</span>
95              </div>
96            </div>
97          </div>
98        </div>
99
100       <div v-if="pastTournaments.length === 0" class="empty-state">
101         <p>Прошедших турниров пока нет.</p>
102       </div>
103     </div>
104
105     <div class="features-section">
106       <h2 class="section-title">Почему выбирают нас</h2>
107       <div class="features-grid">
108         <div class="feature-card">
109           <div class="feature-number">01</div>
110           <h3>LAN финалы</h3>
111           <p>Лучшие команды встречаются на LAN турнирах с полным покрытием расходов</p>
112         </div>
113
114         <div class="feature-card">
115           <div class="feature-number">02</div>
116           <h3>Профессиональная трансляция</h3>
117           <p>Все матчи транслируются с профессиональными комментаторами</p>
118         </div>
119
120         <div class="feature-card">
121           <div class="feature-number">03</div>
122           <h3>Рейтинговая система</h3>
123           <p>Поднимайтесь в рейтинге и получайте инвайты на закрытые турниры</p>
124         </div>
125
126         <div class="feature-card">
127           <div class="feature-number">04</div>
128           <h3>Anti-Cheat защита</h3>
129           <p>Многоуровневая система защиты от нечестной игры</p>
130         </div>
131       </div>
132     </div>
133
134     <div class="prize-section">
135       <div class="prize-content">
136         <h2>СЕЗОН 2026</h2>
137         <div class="prize-amount">$500,000</div>
138         <p>Общий призовой фонд сезона</p>
139         <router-link v-if="!isAuthenticated" to="/register" class="btn btn-primary btn-large">ПРИНЯТЬ УЧАСТИЕ</router-link>
140         <router-link v-else to="/profile" class="btn btn-primary btn-large">МОЙ ПРОФИЛЬ</router-link>
141       </div>
142     </div>
143   </div>
144 </template>
```

```
<script setup>
import { ref, onMounted, onUnmounted, computed } from 'vue';
import { collection, onSnapshot, query, orderBy, limit, startAfter, getDocs } from 'firebase/firestore';
import { db } from '../firebase';
import TournamentCard from './TournamentCard.vue';

const props = defineProps({
  isAuthenticated: { type: Boolean, default: false },
  userName: { type: String, default: '' },
  userEmail: { type: String, default: '' },
  userId: { type: String, default: '' }
});

const tournaments = ref([]);
const displayedCount = ref(15);
const loading = ref(true);
const searchQuery = ref('');
const selectedGame = ref('');
const sortBy = ref('newest');
let unsubscribe = null;

const filteredTournaments = computed(() => {
  let result = tournaments.value.filter(t => t.status === 'active');

  if (selectedGame.value) {
    result = result.filter(t => t.game === selectedGame.value);
  }

  if (searchQuery.value) {
    const query = searchQuery.value.toLowerCase();
    result = result.filter(t =>
      t.name.toLowerCase().includes(query) ||
      (t.description && t.description.toLowerCase().includes(query))
    );
  }

  if (sortBy.value === 'prize-high') {
    result.sort((a, b) => (b.prizePool || 0) - (a.prizePool || 0));
  } else if (sortBy.value === 'prize-low') {
    result.sort((a, b) => (a.prizePool || 0) - (b.prizePool || 0));
  } else if (sortBy.value === 'newest') {
    result.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
  }

  return result;
});

const displayedTournaments = computed(() => {
  return filteredTournaments.value.slice(0, displayedCount.value);
});

const hasMoreTournaments = computed(() => {
  return displayedCount.value < filteredTournaments.value.length;
});

const pastTournaments = computed(() => {
  let result = tournaments.value.filter(t => t.status === 'completed');

  if (selectedGame.value) {
    result = result.filter(t => t.game === selectedGame.value);
  }

  if (searchQuery.value) {
    const query = searchQuery.value.toLowerCase();
    result = result.filter(t =>
      t.name.toLowerCase().includes(query) ||
      (t.description && t.description.toLowerCase().includes(query))
    );
  }

  result.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));

  return result;
});

const loadMore = () => {
  displayedCount.value += 10;
};

const setupRealtimeListener = () => {
  try {
    const q = query(collection(db, 'tournaments'), orderBy('createdAt', 'desc'));

    unsubscribe = onSnapshot(q, (querySnapshot) => {
      tournaments.value = querySnapshot.docs.map(doc => ({
        id: doc.id,
        ...doc.data()
      }));

      loading.value = false;
    }, (error) => {
      console.error('Ошибка подписки:', error);
      tournaments.value = [];
      loading.value = false;
    });
  } catch (error) {
    console.error('Ошибка при инициализации:', error);
    tournaments.value = [];
    loading.value = false;
```

```
225 const setupRealtimeListener = () => {
226   try {
227     const q = query(collection(db, 'tournaments'), orderBy('createdAt', 'desc'));
228
229     unsubscribe = onSnapshot(q, (querySnapshot) => {
230       tournaments.value = querySnapshot.docs.map(doc => ({
231         id: doc.id,
232         ...doc.data()
233       }));
234
235       loading.value = false;
236     }, (error) => {
237       console.error('Ошибка подписки:', error);
238       tournaments.value = [];
239       loading.value = false;
240     });
241   } catch (error) {
242     console.error('Ошибка при инициализации:', error);
243     tournaments.value = [];
244     loading.value = false;
245   }
246 };
247
248 onMounted(() => {
249   setupRealtimeListener();
250 });
251
252 onUnmounted(() => {
253   if (unsubscribe) unsubscribe();
254 });
255 </script>
```

```
1   <template>
2     <div v-if="isAdmin" class="admin-container">
3       <div class="admin-header">
4         <h1>Панель администратора</h1>
5         <p class="admin-subtitle">Управление платформой CYBERTOUR</p>
6       </div>
7
8       <div class="admin-tabs">
9         <button
10          v-for="tab in tabs"
11          :key="tab"
12          :class="{ active: activeTab === tab }"
13          @click="activeTab = tab"
14          class="tab-btn"
15        >
16          {{
17            tab === 'tournaments' ? 'Турниры' :
18            tab === 'registrations' ? 'Заявки' :
19            tab === 'users' ? 'Пользователи' : 'Статистика'
20          }}
21        </button>
22      </div>
23
24      <div v-if="activeTab === 'tournaments'" class="tab-content">
25        <div class="section-header">
26          <h2>Управление турнирами</h2>
27          <button @click="showAddTournament = true" class="btn-add">+ Добавить турнир</button>
28        </div>
29
30        <div class="tournaments-list">
31          <div v-if="loading" class="empty-state">Загрузка данных...</div>
32          <div v-else-if="tournaments.length === 0" class="empty-state">Турниры не найдены</div>
33          <div v-else class="table-wrapper">
34            <table class="admin-table">
35              <thead>
36                <tr>
37                  <th>Название</th>
38                  <th>Игра</th>
39                  <th>Призовой</th>
40                  <th>Участники</th>
41                  <th>Статус</th>
42                  <th>Действия</th>
43                </tr>
44              </thead>
45              <tbody>
46                <tr v-for="t in tournaments" :key="t.id">
47                  <td>{{ t.name }}</td>
48                  <td>{{ t.game }}</td>
49                  <td>${{ t.prizePool?.toLocaleString() }}</td>
50                  <td>{{ t.registeredTeams || 0 }}/{{ t.maxTeams }}</td>
51                  <td>
52                    <span :class="['status-badge', t.status || 'active']">
53                      {{ getStatusText(t.status) }}
54                    </span>
55                  </td>
56                  <td class="actions">
57                    <button @click="editTournament(t)" class="btn-edit">Редактировать</button>
58                    <button @click="deleteTournament(t.id)" class="btn-delete">Удалить</button>
59                  </td>
60                </tr>
61              </tbody>
62            </table>
63          </div>
64        </div>
65      </div>
66
67      <div v-if="activeTab === 'registrations'" class="tab-content">
68        <div class="section-header">
69          <h2>Заявки на участие</h2>
70        </div>
71
72        <div class="registrations-list">
73          <div v-if="loading" class="empty-state">Загрузка заявок...</div>
74          <div v-else-if="allRegistrations.length === 0" class="empty-state">Новых заявок пока нет</div>
75          <div v-else class="table-wrapper">
76            <table class="admin-table">
77              <thead>
78                <tr>
79                  <th>Игрок</th>
80                  <th>Турнир</th>
81                  <th>Дата</th>
82                  <th>Статус</th>
83                  <th>Действия</th>
84                </tr>
85              </thead>
86              <tbody>
87                <tr v-for="reg in allRegistrations" :key="reg.id">
88                  <td>
89                    <div class="user-cell">
90                      <strong>{{ reg.userName || 'Аноним' }}</strong>
91                      <small>ID: {{ reg.userId?.slice(0,8) }}</small>
92                    </div>
93                  </td>
94                  <td>{{ reg.tournamentName }}</td>
95                  <td>{{ formatDate(reg.createdAt) }}</td>
96                  <td>
97                    <span :class="['status-badge', reg.status]">
98                      {{ reg.status === 'pending' ? 'Ожидает' : 'Одобрена' }}
```

```
97                <span :class="['status-badge', reg.status]">
98                  {{ reg.status === 'pending' ? 'Ожидает' : 'Одобрена' }}
99                </span>
100             </td>
101             <td>
102               <div class="actions">
103                 <button
104                   v-if="reg.status === 'pending'"
105                   @click="approveRegistration(reg)"
106                   class="btn-edit"
107                   style="background: #28a745;"
108                 >Одобрить</button>
109                 <button @click="rejectRegistration(reg)" class="btn-delete">Удалить</button>
110               </div>
111             </td>
112           </tr>
113         </tbody>
114       </table>
115     </div>
116   </div>
117 </div>
118
119 <div v-if="activeTab === 'users'" class="tab-content">
120   <div class="section-header"><h2>Пользователи системы</h2></div>
121   <div v-if="loading" class="empty-state">Загрузка...</div>
122   <div v-else class="table-wrapper">
123     <table class="admin-table">
124       <thead>
125         <tr>
126           <th>Email</th>
127           <th>Роль</th>
128           <th>Действия</th>
129         </tr>
130       </thead>
131       <tbody>
132         <tr v-for="user in users" :key="user.id">
133           <td>{{ user.email }}</td>
134           <td><span :class="['role-badge', user.role]">{{ user.role || 'user' }}</span></td>
135           <td class="actions">
136             <button @click="deleteUser(user.id)" class="btn-delete" :disabled="user.role === 'admin'">Удалить</button>
137           </td>
138         </tr>
139       </tbody>
140     </table>
141   </div>
142 </div>
143
144 <div v-if="activeTab === 'stats'" class="tab-content">
145   <div class="stats-grid">
146     <div class="stat-card">
147       <h3>Всего игроков</h3>
148       <p class="stat-number">{{ users.length }}</p>
149     </div>
150     <div class="stat-card">
151       <h3>Активных турниров</h3>
152       <p class="stat-number">{{ tournaments.length }}</p>
153     </div>
154     <div class="stat-card">
155       <h3>Всего заявок</h3>
156       <p class="stat-number">{{ allRegistrations.length }}</p>
157     </div>
158   </div>
159 </div>
160
161 <div v-if="showAddTournament" class="modal-overlay" @click="showAddTournament = false">
162   <div class="modal-content" @click.stop>
163     <h3>Новый турнир</h3>
164     <form @submit.prevent="addTournament" class="tournament-form">
165       <input v-model="newTournament.name" placeholder="Название" required />
166       <input v-model="newTournament.game" placeholder="Игра (CS2, Dota...)" required />
167       <input v-model.number="newTournament.prizePool" type="number" placeholder="Призовой фонд" required />
168       <input v-if="newTournament.status !== 'completed'" v-model.number="newTournament.maxTeams" type="number" placeholder="Макс. команд" required />
169       <input v-model="newTournament.startDate" type="date" required />
170       <select v-model="newTournament.status" class="status-select" required>
171         <option value="active">Активный турнир</option>
172         <option value="completed">Завершенный турнир</option>
173       </select>
174       <input v-if="newTournament.status === 'completed'" v-model="newTournament.winner" placeholder="Победитель" />
175       <div class="form-actions">
176         <button type="submit" class="btn-save">Создать</button>
177         <button type="button" class="btn-cancel" @click="showAddTournament = false">Отмена</button>
178       </div>
179     </form>
180   </div>
181 </div>
182
183 <div v-if="showEditTournament" class="modal-overlay" @click="showEditTournament = false">
184   <div class="modal-content" @click.stop>
185     <h3>Редактировать турнир</h3>
186     <form @submit.prevent="updateTournament" class="tournament-form">
187       <input v-model="editingTournament.name" placeholder="Название" required />
188       <input v-model="editingTournament.game" placeholder="Игра (CS2, Dota...)" required />
189       <input v-model.number="editingTournament.prizePool" type="number" placeholder="Призовой фонд" required />
```

```
183       <div v-if="showEditTournament" class="modal-overlay" @click="showEditTournament = false">
184         <div class="modal-content" @click.stop>
185           <h3>Редактировать турнир</h3>
186           <form @submit.prevent="updateTournament" class="tournament-form">
187             <input v-model="editingTournament.name" placeholder="Название" required />
188             <input v-model="editingTournament.game" placeholder="Игра (CS2, Dota...)" required />
189             <input v-model.number="editingTournament.prizePool" type="number" placeholder="Призовой фонд" required />
190             <input v-if="editingTournament.status !== 'completed'" v-model.number="editingTournament.maxTeams" type="number" placeholder="Макс. команд" required />
191             <input v-model="editingTournament.startDate" type="date" required />
192             <select v-model="editingTournament.status" class="status-select">
193               <option value="active">Активный</option>
194               <option value="completed">Завершен</option>
195             </select>
196             <input v-if="editingTournament.status === 'completed'" v-model="editingTournament.winner" placeholder="Победитель" />
197             <div class="form-actions">
198               <button type="submit" class="btn-save">Сохранить</button>
199               <button type="button" class="btn-cancel" @click="showEditTournament = false">Отмена</button>
200             </div>
201           </form>
202         </div>
203       </div>
204     </div>
205
206     <div v-else class="denied-container">
207       <h2>403 - Доступ запрещен</h2>
208       <p>У вас недостаточно прав для просмотра этой страницы.</p>
209       <router-link to="/">Вернуться на главную</router-link>
210     </div>
211   </template>
212
213   <script setup>
214   import { ref, onMounted, watch } from 'vue';
215   import {
216     collection, getDocs, doc, deleteDoc, addDoc,
217     updateDoc, query, orderBy, increment
218   } from 'firebase/firestore';
219   import { db } from '../firebase';
220
221   const props = defineProps({
222     isAdmin: [String, Boolean],
223     isAuthenticated: Boolean,
224     userId: String,
225     userName: String
226   });
227
228   const loading = ref(true);
229   const activeTab = ref('tournaments');
230   const tabs = ['tournaments', 'registrations', 'users', 'stats'];
231
232   const tournaments = ref([]);
233   const users = ref([]);
234   const allRegistrations = ref([]);
235   const showAddTournament = ref(false);
236   const showEditTournament = ref(false);
237   const editingTournament = ref(null);
238
239   const newTournament = ref({
240     name: '', game: '', prizePool: 0, maxTeams: 16, startDate: '', status: 'active', winner: ''
241   });
242
243   const loadData = async () => {
244     if (!props.isAdmin) return;
245     loading.value = true;
246     try {
247       const [tSnap, uSnap, rSnap] = await Promise.all([
248         getDocs(collection(db, 'tournaments')),
249         getDocs(collection(db, 'users')),
250         getDocs(query(collection(db, 'registrations'), orderBy('createdAt', 'desc')))
251       ]);
252
253       tournaments.value = tSnap.docs.map(d => ({ id: d.id, ...d.data() }));
254       users.value = uSnap.docs.map(d => ({ id: d.id, ...d.data() }));
255       allRegistrations.value = rSnap.docs.map(d => ({ id: d.id, ...d.data() }));
256     } catch (e) {
257       console.error("Ошибка загрузки данных:", e);
258     } finally {
259       loading.value = false;
260     }
261   };
262
263   watch(() => props.isAdmin, (newVal) => {
264     if (newVal) loadData();
265   }, { immediate: true });
266
267   const addTournament = async () => {
268     try {
269       loading.value = true;
```

```
267  const addTournament = async () => {
268    try {
269      loading.value = true;
270      await addDoc(collection(db, "tournaments"), {
271        ...newTournament.value,
272        registeredTeams: 0,
273        createdAt: new Date().toISOString()
274      });
275      showAddTournament.value = false;
276      newTournament.value = { name: '', game: '', prizePool: 0, maxTeams: 16, startDate: '', status: 'active', win
277      await loadData();
278      alert("Турнир создан!");
279    } catch (error) {
280      console.error("Ошибка:", error);
281    } finally {
282      loading.value = false;
283    }
284  };
285
286  const approveRegistration = async (reg) => {
287    try {
288      await updateDoc(doc(db, 'registrations', reg.id), { status: 'approved' });
289      await loadData();
290    } catch (e) {
291      alert('Ошибка при обновлении');
292    }
293  };
294
295  const rejectRegistration = async (reg) => {
296    if (!confirm('Удалить заявку?')) return;
297    try {
298      await deleteDoc(doc(db, 'registrations', reg.id));
299      if (reg.tournamentId) {
300        await updateDoc(doc(db, 'tournaments', reg.tournamentId), {
301          registeredTeams: increment(-1)
302        });
303      }
304      await loadData();
305    } catch (e) {
306      alert('Ошибка при удалении');
307    }
308  };
309
310  const deleteTournament = async (id) => {
311    if (confirm('Удалить турнир?')) {
312      await deleteDoc(doc(db, 'tournaments', id));
313      await loadData();
314    }
315  };
316
317  const editTournament = (tournament) => {
318    editingTournament.value = { ...tournament };
319    showEditTournament.value = true;
320  };
321
322  const updateTournament = async () => {
323    try {
324      loading.value = true;
325      await updateDoc(doc(db, 'tournaments', editingTournament.value.id), {
326        name: editingTournament.value.name,
327        game: editingTournament.value.game,
328        prizePool: editingTournament.value.prizePool,
329        maxTeams: editingTournament.value.maxTeams,
330        startDate: editingTournament.value.startDate,
331        status: editingTournament.value.status,
332        winner: editingTournament.value.winner || null
333      });
334      showEditTournament.value = false;
335      editingTournament.value = null;
336      await loadData();
337      alert("Турнир обновлен!");
338    } catch (error) {
339      console.error("Ошибка:", error);
340      alert("Ошибка при обновлении турнира");
341    } finally {
342      loading.value = false;
343    }
344  };
345
346  const deleteUser = async (id) => {
347    if (confirm('Удалить пользователя?')) {
348      await deleteDoc(doc(db, 'users', id));
349      await loadData();
350    }
351  };
352
353  const formatDate = (date) => {
354    if (!date) return '---';
355    const d = date.toDate ? date.toDate() : new Date(date);
356    return d.toLocaleDateString();
357  };
358
359  const getStatusText = (status) => {
360    return status === 'completed' ? 'Завершен' : 'Активный';
361  };
362  </script>
```

Приложение 6 – Login.vue

```
1   <template>
2     <div class="auth-container">
3       <div class="auth-card">
4         <h2 class="auth-title">ВХОД В СИСТЕМУ</h2>
5         <p class="auth-subtitle">Доступ к управлению турнирами</p>
6
7         <form @submit.prevent="handleLogin" class="auth-form">
8           <div class="form-group">
9             <input
10              v-model="email"
11              type="email"
12              placeholder="Email"
13              required
14            />
15          </div>
16
17          <div class="form-group">
18            <input
19              v-model="password"
20              type="password"
21              placeholder="Пароль"
22              required
23            />
24          </div>
25
26          <div v-if="errorMessage" class="error-message">
27            {{ errorMessage }}
28          </div>
29
30          <button
31            type="submit"
32            class="btn-submit"
33            :disabled="loading"
34          >
35            {{ loading ? 'ВХОД...' : 'ВОЙТИ' }}
36          </button>
37        </form>
38
39        <p class="auth-footer">
40          Нет аккаунта?
41          <router-link to="/register">Зарегистрироваться</router-link>
42        </p>
43      </div>
44    </div>
45  </template>
46
47  <script setup>
48  import { ref } from "vue";
49  import { signInWithEmailAndPassword } from "firebase/auth";
50  import { auth } from "../firebase";
51  import { useRouter } from "vue-router";
52
53  const router = useRouter();
54  const email = ref("");
55  const password = ref("");
56  const loading = ref(false);
57  const errorMessage = ref("");
58
59  const handleLogin = async () => {
60    errorMessage.value = "";
61
62    if (!email.value || !password.value) {
63      errorMessage.value = "Пожалуйста, заполните все поля";
64      return;
65    }
66
67    loading.value = true;
68
69    try {
70      await signInWithEmailAndPassword(auth, email.value, password.value);
71      router.push("/");
72    } catch (error) {
73      switch (error.code) {
74        case 'auth/invalid-credential':
75        case 'auth/user-not-found':
76        case 'auth/wrong-password':
77          errorMessage.value = "Неверный email или пароль";
78          break;
79        case 'auth/invalid-email':
80          errorMessage.value = "Некорректный email адрес";
81          break;
82        default:
83          errorMessage.value = "Ошибка при входе. Попробуйте позже";
84      }
85    } finally {
86      loading.value = false;
87    }
88  };
89  </script>
```

Приложение 7 – Register.vue

```
1   <template>
2     <div class="auth-container">
3       <div class="auth-card">
4         <h2 class="auth-title">РЕГИСТРАЦИЯ</h2>
5         <p class="auth-subtitle">Станьте частью киберспортивного сообщества</p>
6
7         <form @submit.prevent="handleRegister" class="auth-form">
8           <div class="form-group">
9             <input
10              v-model="email"
11              type="email"
12              placeholder="Email"
13              required
14            />
15          </div>
16
17          <div class="form-group">
18            <input
19              v-model="displayName"
20              type="text"
21              placeholder="Ваше имя"
22              required
23            />
24          </div>
25
26          <div class="form-group">
27            <input
28              v-model="password"
29              type="password"
30              placeholder="Пароль (минимум 6 символов)"
31              required
32              minlength="6"
33            />
34          </div>
35
36          <div class="form-group">
37            <input
38              v-model="confirmPassword"
39              type="password"
40              placeholder="Подтвердите пароль"
41              required
42            />
43          </div>
44
45          <div v-if="errorMessage" class="error-message">
46            {{ errorMessage }}
47          </div>
48
49          <button
50            type="submit"
51            class="btn-submit"
52            :disabled="loading"
53          >
54            {{ loading ? 'РЕГИСТРАЦИЯ...' : 'СОЗДАТЬ АККАУНТ' }}
55          </button>
56        </form>
57
58        <p class="auth-footer">
59          Уже есть аккаунт?
60          <router-link to="/login">Войти</router-link>
61        </p>
62      </div>
63    </div>
64  </template>
65
66  <script setup>
67  import { ref } from "vue";
68  import { auth, db } from "../firebase";
69  import { createUserWithEmailAndPassword, updateProfile } from "firebase/auth";
70  import { doc, setDoc } from "firebase/firestore";
71  import { useRouter } from "vue-router";
72
73  const router = useRouter();
74  const email = ref("");
75  const displayName = ref("");
76  const password = ref("");
77  const confirmPassword = ref("");
78  const errorMessage = ref("");
79  const loading = ref(false);
80
81  const handleRegister = async () => {
82    errorMessage.ref = "";
83
84    if (password.value !== confirmPassword.value) {
85      errorMessage.value = "Пароли не совпадают";
86      return;
87    }
88
89    loading.value = true;
90
91    try {
92      const userCredential = await createUserWithEmailAndPassword(
93        auth,
94        email.value,
95        password.value
96      );
97
98      const user = userCredential.user;
```

```
91    try {
92      const userCredential = await createUserWithEmailAndPassword(
93        auth,
94        email.value,
95        password.value
96      );
97
98      const user = userCredential.user;
99
100     await updateProfile(user, {
101       displayName: displayName.value
102     });
103
104     await setDoc(doc(db, "users", user.uid), {
105       uid: user.uid,
106       displayName: displayName.value,
107       email: email.value,
108       role: "user",
109       createdAt: new Date().toISOString()
110     });
111
112     console.log("Регистрация успешна!");
113     router.push("/");
114
115   } catch (error) {
116     console.error("Ошибка при регистрации:", error);
117     if (error.code === 'auth/email-already-in-use') {
118       errorMessage.value = "Этот Email уже используется";
119     } else {
120       errorMessage.value = "Произошла ошибка при создании аккаунта";
121     }
122   } finally {
123     loading.value = false;
124   }
125 };
126 </script>
127
```

Приложение 8 – Profile.vue

```
1   <template>
2     <div class="profile-container">
3       <div class="profile-header">
4         <div class="profile-main-info">
5           <div class="avatar">{{ userInitials }}</div>
6           <div class="details">
7             <div v-if="!isEditing" class="name-display">
8               <h1>{{ userName || 'Боец' }}</h1>
9               <button @click="startEditing" class="btn-edit-inline">⚙ Изменить имя</button>
10            </div>
11
12            <div v-else class="name-edit-form">
13              <input
14                v-model="editName"
15                type="text"
16                class="edit-input"
17                placeholder="Введите новое имя"
18                @keyup.enter="saveName"
19              />
20              <div class="edit-actions">
21                <button @click="saveName" class="btn-save-small" :disabled="saving">Сохранить</button>
22                <button @click="isEditing = false" class="btn-cancel-small">Отмена</button>
23              </div>
24            </div>
25            <p class="user-email">{{ userEmail }}</p>
26          </div>
27        </div>
28      </div>
29
30      <div class="profile-content">
31        <h3>Мои турниры</h3>
32        <div v-if="loading" class="loading">Загрузка ваших заявок...</div>
33
34        <div v-else-if="registrations.length === 0" class="empty-state">
35          <p>Вы еще не записались ни на один турнир.</p>
36          <router-link to="/" class="btn-link">Найти турнир</router-link>
37        </div>
38
39        <div v-else class="reg-list">
40          <div v-for="reg in registrations" :key="reg.id" class="reg-card">
41            <div class="reg-info">
42              <h4>{{ reg.tournamentName }}</h4>
43              <span class="game-tag">{{ reg.game }}</span>
44            </div>
45            <div class="reg-status" :class="reg.status">
46              {{ reg.status === 'pending' ? 'На рассмотрении' : 'Подтвержден' }}
47            </div>
48          </div>
49        </div>
50      </div>
51    </div>
52  </template>
```

```
<script setup>
import { ref, onMounted, computed, watch } from 'vue';
import { db } from '../firebase';
import { collection, query, where, getDocs, doc, updateDoc } from 'firebase/firestore';

const props = defineProps({
  userId: String,
  userEmail: String,
  userName: String
});

const registrations = ref([]);
const loading = ref(true);
const isEditing = ref(false);
const editName = ref('');
const saving = ref(false);

const userInitials = computed(() => {
  return props.userName ? props.userName.charAt(0).toUpperCase() : '?';
});

const startEditing = () => {
  editName.value = props.userName;
  isEditing.value = true;
};

const saveName = async () => {
  if (!editName.value.trim() || editName.value === props.userName) {
    isEditing.value = false;
    return;
  }

  saving.value = true;
  try {
    const userRef = doc(db, 'users', props.userId);
    await updateDoc(userRef, {
      displayName: editName.value
    });

    window.location.reload();
  } catch (e) {
    console.error("Ошибка при сохранении имени:", e);
    alert("Не удалось сохранить имя.");
  } finally {
    saving.value = false;
    isEditing.value = false;
  }
};

const loadMyRegistrations = async () => {
  if (!props.userId) return;
  loading.value = true;
  try {
    const q = query(
      collection(db, 'registrations'),
      where('userId', '==', props.userId)
    );
    const snap = await getDocs(q);
    registrations.value = snap.docs.map(doc => ({ id: doc.id, ...doc.data() }));
  } catch (e) {
    console.error("Ошибка загрузки регистраций:", e);
  } finally {
    loading.value = false;
  }
};

onMounted(() => {
  if (props.userId) loadMyRegistrations();
});

watch(() => props.userId, (newId) => {
  if (newId) loadMyRegistrations();
});
</script>
```

Приложение 9 – TournamentCard.vue

```
1  <template>
2    <div class="tournament-card">
3      <div class="tournament-header">
4        <div class="tournament-game">{{ tournament.game }}</div>
5      </div>
6
7      <router-link :to="`/tournament/${tournament.id}`" class="tournament-name-link">
8        <h3 class="tournament-name">{{ tournament.name }}</h3>
9      </router-link>
10
11     <div class="tournament-meta">
12       <span>💰 ${{ tournament.prizePool?.toLocaleString() || 0 }}</span>
13       <span v-if="tournament.status !== 'completed'" :class="{ 'text-full': isFull(tournament) }">
14         Команды: {{ tournament.registeredTeams || 0 }}/{{ tournament.maxTeams || 0 }}
15       </span>
16     </div>
17
18     <div class="tournament-status" :class="getStatusClass(tournament)">
19       {{ getStatus(tournament) }}
20     </div>
21
22     <div v-if="tournament.status === 'completed' && tournament.winner" class="tournament-winner">
23       <span class="winner-label">Победитель:</span>
24       <span class="winner-name">{{ tournament.winner }}</span>
25     </div>
26
27     <div class="tournament-actions">
28       <button
29         v-if="isAuthenticated && tournament.status !== 'completed'"
30         @click="handleRegister"
31         class="btn-register"
32         :disabled="registering || isFull(tournament)"
33       >
34         <span v-if="registering">ОБРАБОТКА...</span>
35         <span v-else-if="isFull(tournament)">МЕСТ НЕТ</span>
36         <span v-else>УЧАСТВОВАТЬ</span>
37       </button>
38       <router-link v-if="!isAuthenticated" to="/login" class="btn-register">ВОЙТИ</router-link>
39     </div>
40   </div>
41 </template>
42
```

```
<script setup>
import { ref } from 'vue';
import { db } from '../firebase';
import {
  collection,
  addDoc,
  doc,
  updateDoc,
  increment,
  serverTimestamp
} from 'firebase/firestore';

const props = defineProps({
  tournament: Object,
  isAuthenticated: Boolean,
  userId: String,
  userName: String
});

const emit = defineEmits(['registered']);
const registering = ref(false);

const handleRegister = async () => {
  if (registering.value) return;

  const max = Number(props.tournament.maxTeams) || 16;
  const current = Number(props.tournament.registeredTeams) || 0;

  if (current >= max) {
    alert('Ошибка: Места в этом турнире действительно закончились!');
    return;
  }

  registering.value = true;

  try {
    await addDoc(collection(db, 'registrations'), {
      tournamentId: props.tournament.id,
      tournamentName: props.tournament.name,
      game: props.tournament.game,
      userId: props.userId,
      userName: props.userName || 'Аноним',
      status: 'pending',
      createdAt: serverTimestamp()
    });

    const tournamentRef = doc(db, 'tournaments', props.tournament.id);
    await updateDoc(tournamentRef, {
      registeredTeams: increment(1)
    });

    alert('Успешная регистрация!');
    emit('registered');

  } catch (e) {
    console.error("ОШИБКА ФАЙРБЕЙСА:", e.code, e.message);
    if (e.code === 'permission-denied') {
      alert('Ошибка доступа: Проверьте правила безопасности Firebase!');
    } else {
      alert('Произошла ошибка при регистрации. Попробуйте позже.');
    }
  } finally {
    registering.value = false;
  }
};

const getStatus = (t) => {
  if (t.status === 'completed') return 'ЗАВЕРШЕН';
  return t.registeredTeams >= t.maxTeams ? 'РЕГИСТРАЦИЯ ЗАКРЫТА' : 'ИДЕТ НАБОР';
};

const getStatusClass = (t) => {
  if (t.status === 'completed') return 'status-completed';
  return t.registeredTeams >= t.maxTeams ? 'status-closed' : 'status-open';
};
const isFull = (t) => (t.registeredTeams || 0) >= (t.maxTeams || 0);

const formatDate = (date) => {
  if (!date) return '';
  return new Date(date).toLocaleDateString();
};
</script>
```

Приложение 10 – TournamentDetail.vue

```
1   <template>
2     <div class="detail-container">
3       <div v-if="loading" class="loading-spinner">
4         <p>Загрузка информации о турнире...</p>
5       </div>
6
7       <div v-else-if="!tournament" class="error-state">
8         <h2>Турнир не найден</h2>
9         <router-link to="/" class="btn btn-primary">Вернуться на главную</router-link>
10      </div>
11
12      <div v-else class="tournament-detail">
13        <!-- Хедер -->
14        <div class="detail-header">
15          <div class="header-content">
16            <div class="breadcrumb">
17              <router-link to="/">Главная</router-link>
18              <span>/</span>
19              <span>{{ tournament.game }}</span>
20            </div>
21            <h1 class="detail-title">{{ tournament.name }}</h1>
22            <p class="detail-subtitle">{{ tournament.description || 'Эпический киберспортивный турнир' }}</p>
23          </div>
24        </div>
25
26        <div class="detail-content">
27          <div class="main-section">
28            <div class="status-banner">
29              <div class="status-item">
30                <span class="label">Статус:</span>
31                <span :class="['status-badge', getStatusClass(tournament)]">
32                  {{ getStatus(tournament) }}
33                </span>
34              </div>
35              <div class="status-item">
36                <span class="label">Участников:</span>
37                <span class="value">{{ tournament.registeredTeams || 0 }}/{{ tournament.maxTeams || 16 }}</span>
38              </div>
39              <div class="status-item">
40                <span class="label">Прогресс:</span>
41                <div class="progress-bar">
42                  <div class="progress-fill" :style="{ width: getProgressPercent(tournament) + '%' }"></div>
43                </div>
44              </div>
45            </div>
46
47            <div v-if="tournament.status === 'completed' && tournament.winner" class="winner-banner">
48              <div class="winner-content">
49                <div class="trophy-icon">🏆</div>
50                <div class="winner-info">
51                  <h3>Победитель турнира</h3>
52                  <p class="winner-name">{{ tournament.winner }}</p>
53                </div>
54              </div>
55            </div>
56
57            <div class="action-section">
58              <button
59                v-if="isAuthenticated"
60                @click="handleRegister"
61                class="btn btn-register"
62                :disabled="registering || isFull(tournament) || isRegistered"
63              >
64                <span v-if="registering">ОБРАБОТКА...</span>
65                <span v-else-if="isRegistered">ВЫ ЗАРЕГИСТРИРОВАНЫ</span>
66                <span v-else-if="isFull(tournament)">МЕСТ НЕТ</span>
67                <span v-else>УЧАСТВОВАТЬ В ТУРНИРЕ</span>
68              </button>
69              <router-link v-else to="/login" class="btn btn-register">ВОЙТИ ДЛЯ ЗАПИСИ</router-link>
70            </div>
71
72            <div class="info-grid">
73              <div class="info-card">
74                <div class="info-body">
75                  <h4>Призовой фонд</h4>
76                  <p class="prize-amount">$${{ (tournament.prizePool || 0).toLocaleString() }}</p>
77                </div>
78              </div>
79
80              <div class="info-card">
81                <div class="info-body">
82                  <h4>Дата старта</h4>
83                  <p>{{ formatDate(tournament.startDate) }}</p>
84                </div>
85              </div>
86
87              <div class="info-card">
88                <div class="info-body">
89                  <h4>Регистрация до</h4>
90                  <p>{{ formatDate(tournament.registrationDeadline) }}</p>
91                </div>
92              </div>
93
```

```
            <div class="info-card">
              <div class="info-body">
                <h4>Формат</h4>
                <p>{{ tournament.format || 'Best of 3 (BO3)' }}</p>
              </div>
            </div>
          </div>
        </div>

        <aside class="sidebar">
          <div class="sidebar-card">
            <h3>О турнире</h3>
            <div class="info-list">
              <div class="info-item">
                <span class="key">Игра:</span>
                <span class="value">{{ tournament.game }}</span>
              </div>
              <div class="info-item">
                <span class="key">Команд:</span>
                <span class="value">{{ tournament.maxTeams || 16 }}</span>
              </div>
              <div class="info-item">
                <span class="key">Уровень:</span>
                <span class="value">{{ tournament.level || 'Любительский' }}</span>
              </div>
              <div class="info-item">
                <span class="key">Регион:</span>
                <span class="value">{{ tournament.region || 'Online' }}</span>
              </div>
            </div>
          </div>

          <div class="sidebar-card">
            <h3>⭐ Оценка турнира</h3>
            <div class="rating">
              <div class="stars">
                <span v-for="i in 5" :key="i" class="star">★</span>
              </div>
              <p class="rating-text">4.8/5 (245 оценок)</p>
            </div>
          </div>

          <div class="sidebar-card">
            <h3>👤 Организатор</h3>
            <div class="organizer">
              <div class="organizer-avatar">🏆</div>
              <div class="organizer-info">
                <h4>{{ tournament.organizer || 'CYBERTOUR' }}</h4>
                <p>Проверенный организатор</p>
              </div>
            </div>
          </div>
        </aside>
      </div>

      <div v-if="relatedTournaments.length > 0" class="related-section">
        <h2>🎮 Похожие турниры</h2>
        <div class="related-grid">
          <TournamentCard
            v-for="t in relatedTournaments"
            :key="t.id"
            :tournament="t"
            :is-authenticated="isAuthenticated"
            :user-id="userId"
            :user-name="userName"
          />
        </div>
      </div>

      <div class="back-button-container">
        <router-link to="/" class="btn btn-secondary">← Вернуться к каталогу</router-link>
      </div>
    </div>
  </div>
</template>
```

```
<script setup>
import { ref, onMounted, computed, onUnmounted } from 'vue';
import { useRoute } from 'vue-router';
import { collection, query, where, getDocs, addDoc, serverTimestamp } from 'firebase/firestore';
import { db } from '../firebase';
import { getTournamentById, getRelatedTournaments, subscribeToTournamentReviews } from '../firebaseService';
import TournamentCard from './TournamentCard.vue';

const route = useRoute();
const props = defineProps({
  isAuthenticated: Boolean,
  userId: String,
  userName: String
});

const tournament = ref(null);
const allTournaments = ref([]);
const reviews = ref([]);
const loading = ref(true);
const registering = ref(false);
const isRegistered = ref(false);
let unsubscribeReviews = null;

const tournamentId = route.params.id;

const loadTournament = async () => {
  try {
    tournament.value = await getTournamentById(tournamentId);
    if (tournament.value) {
      checkIfRegistered();
      loadRelatedTournaments();
      subscribeReviews();
    }
  } catch (error) {
    console.error('Ошибка загрузки турнира:', error);
    tournament.value = null;
  } finally {
    loading.value = false;
  }
};

const checkIfRegistered = async () => {
  if (!props.isAuthenticated || !props.userId) return;

  try {
    const q = query(
      collection(db, 'registrations'),
      where('userId', '==', props.userId),
      where('tournamentId', '==', tournamentId)
    );
    const querySnapshot = await getDocs(q);
    isRegistered.value = !querySnapshot.empty;
  } catch (error) {
    console.error('Ошибка проверки регистрации:', error);
  }
};

const loadRelatedTournaments = async () => {
  try {
    if (tournament.value) {
      allTournaments.value = await getRelatedTournaments(tournament.value.game, tournamentId, 3);
    }
  } catch (error) {
    console.error('Ошибка загрузки похожих турниров:', error);
  }
};

const subscribeReviews = () => {
  if (!tournament.value) return;

  const q = query(
    collection(db, 'reviews'),
    where('tournamentId', '==', tournamentId)
  );

  unsubscribeReviews = getDocs(q).then(snapshot => {
    reviews.value = snapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));
  });
};
```

```
const relatedTournaments = computed(() => allTournaments.value);

const handleRegister = async () => {
  if (registering.value || isRegistered.value) return;

  registering.value = true;
  try {
    await addDoc(collection(db, 'registrations'), {
      tournamentId: tournamentId,
      tournamentName: tournament.value.name,
      game: tournament.value.game,
      userId: props.userId,
      userName: props.userName || 'Аноним',
      status: 'pending',
      createdAt: serverTimestamp()
    });

    alert('✅ Успешная регистрация!');
    isRegistered.value = true;
  } catch (error) {
    console.error('Ошибка регистрации:', error);
    alert('❌ Ошибка при регистрации. Попробуйте позже.');
  } finally {
    registering.value = false;
  }
};

const isFull = (t) => (t.registeredTeams || 0) >= (t.maxTeams || 16);

const getStatus = (t) => {
  if (t.status === 'completed') return 'ЗАВЕРШЕН';
  const now = new Date();
  const deadline = new Date(t.registrationDeadline);
  if (now > deadline) return 'РЕГИСТРАЦИЯ ЗАКРЫТА';
  return 'ОТКРЫТА РЕГИСТРАЦИЯ';
};

const getStatusClass = (t) => {
  if (t.status === 'completed') return 'completed';
  return new Date() > new Date(t.registrationDeadline) ? 'closed' : 'open';
};

const getProgressPercent = (t) => {
  const percent = Math.round(((t.registeredTeams || 0) / (t.maxTeams || 16)) * 100);
  return Math.min(percent, 100);
};

const formatDate = (dateString) => {
  if (!dateString) return 'N/A';
  const date = new Date(dateString);
  return date.toLocaleDateString('ru-RU', {
    year: 'numeric',
    month: 'long',
    day: 'numeric',
    hour: '2-digit',
    minute: '2-digit'
  });
};

onMounted(() => {
  loadTournament();
});

onUnmounted(() => {
  if (unsubscribeReviews) unsubscribeReviews();
});
</script>
```